3099704: AI for Digital Health

Generative AI: LLM

Prof. Peerapon Vateekul, Ph.D.
Peerapon.v@chula.ac.th

Outline

- ML Tasks
- Generative AI (Gen AI)
- LLM Techniques
  - 1) Prompting
  - 2) RAG
  - 3) Fine-Tuning
  - 4) Agentic
- LLM Tools
  - 1) LangChain
  - 2) LangGraph
  - 3) LangSmith
  - 4) N8N & Labs

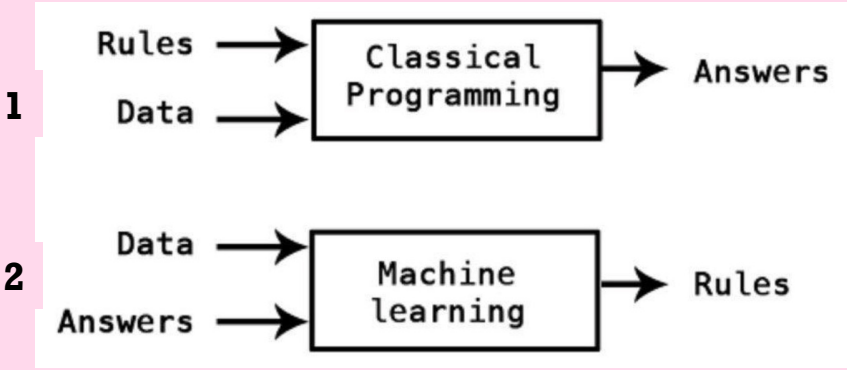
ML Tasks

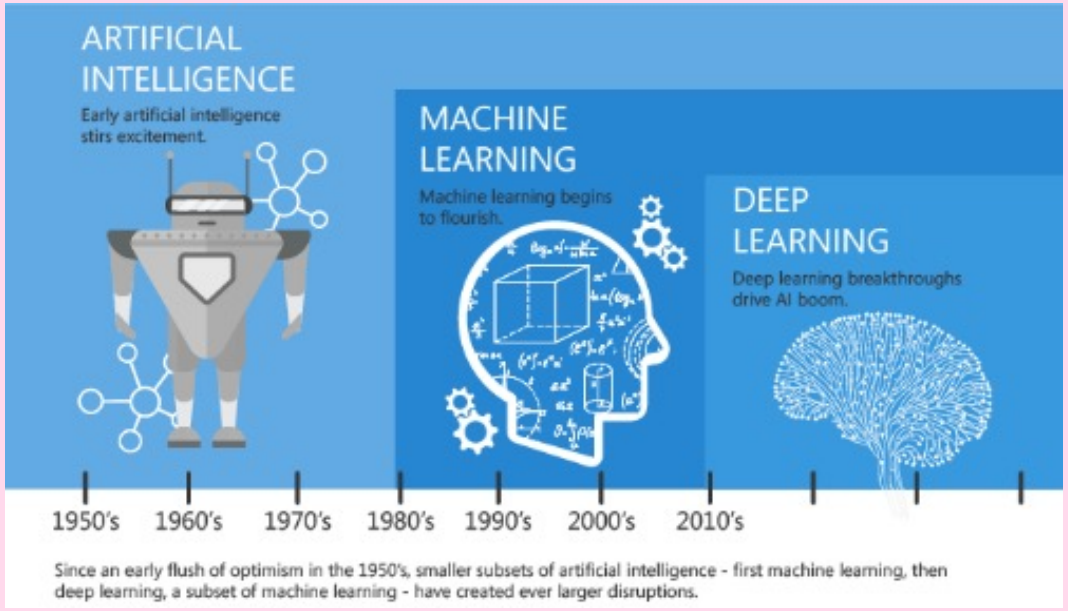
Supervised Learning
Unsupervised Learning
Reinforcement Learning (RL)

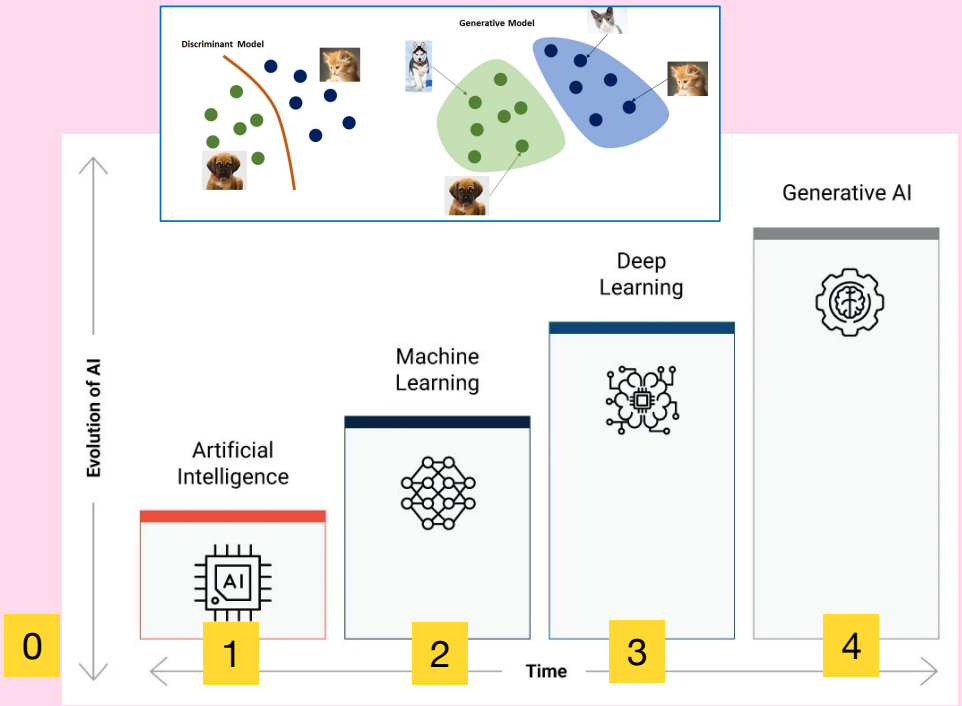
CHULA ENGINEERING COMPUTER
Foundation toward Innovation

AI = Automation

- 0) Not AI Solution (not automatic)
- 1) Rule-based AI
- 2) Machine Learning (ML)

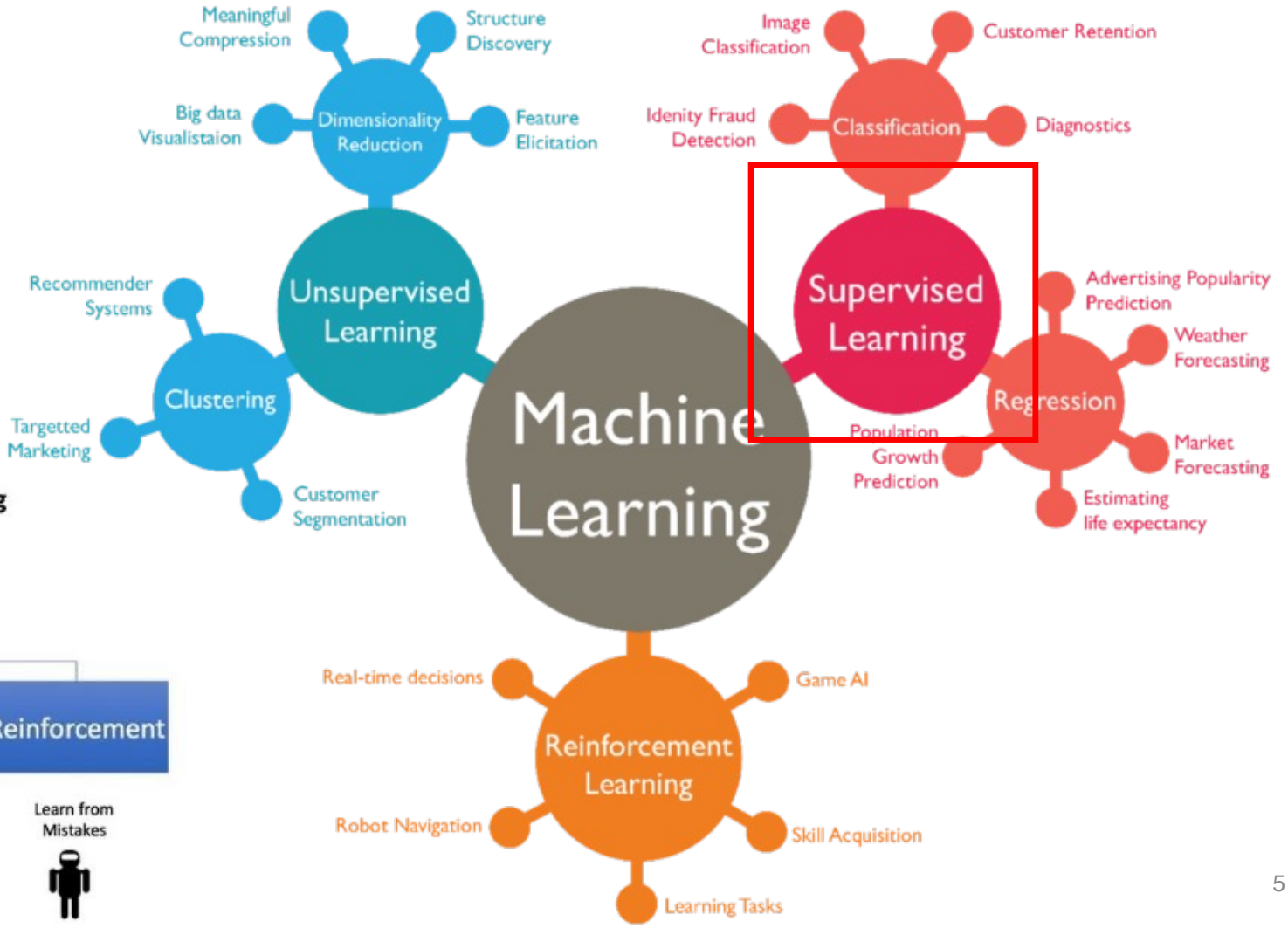




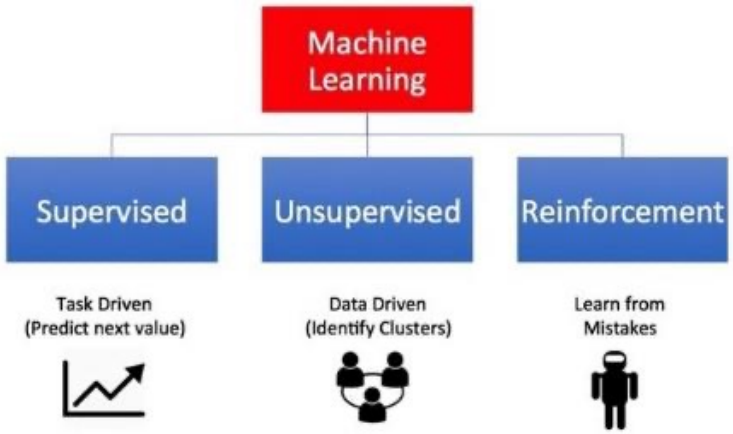


https://mc.ai/machine-learning-basics-artificial-intelligence-machine-learning-and-deep-learning/

Task1: Supervised learning (predictive task)



Types of Machine Learning

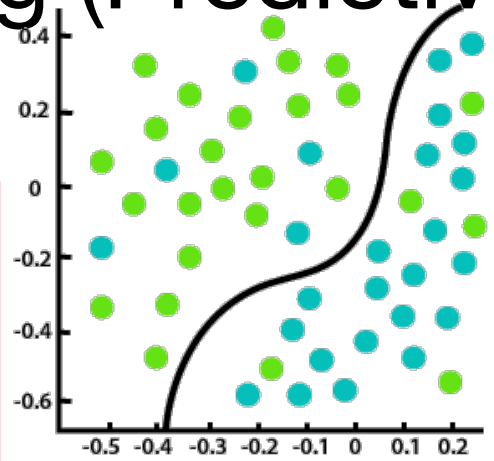


Supervised Learning (Predictive Task)

inputs | target

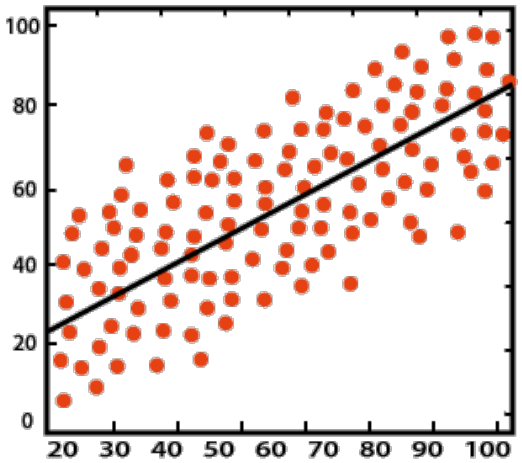
Age	Temp	Gender	Smell	Covid
25	39.0	Female	No	Yes
35	38.9	Female	No	Yes
32	36.5	Male	Yes	No

- Goal: To learn a prediction model mapping from inputs to output.
- Data without label (answer) is meaningless!
- Label should be provided by experts!



Classification

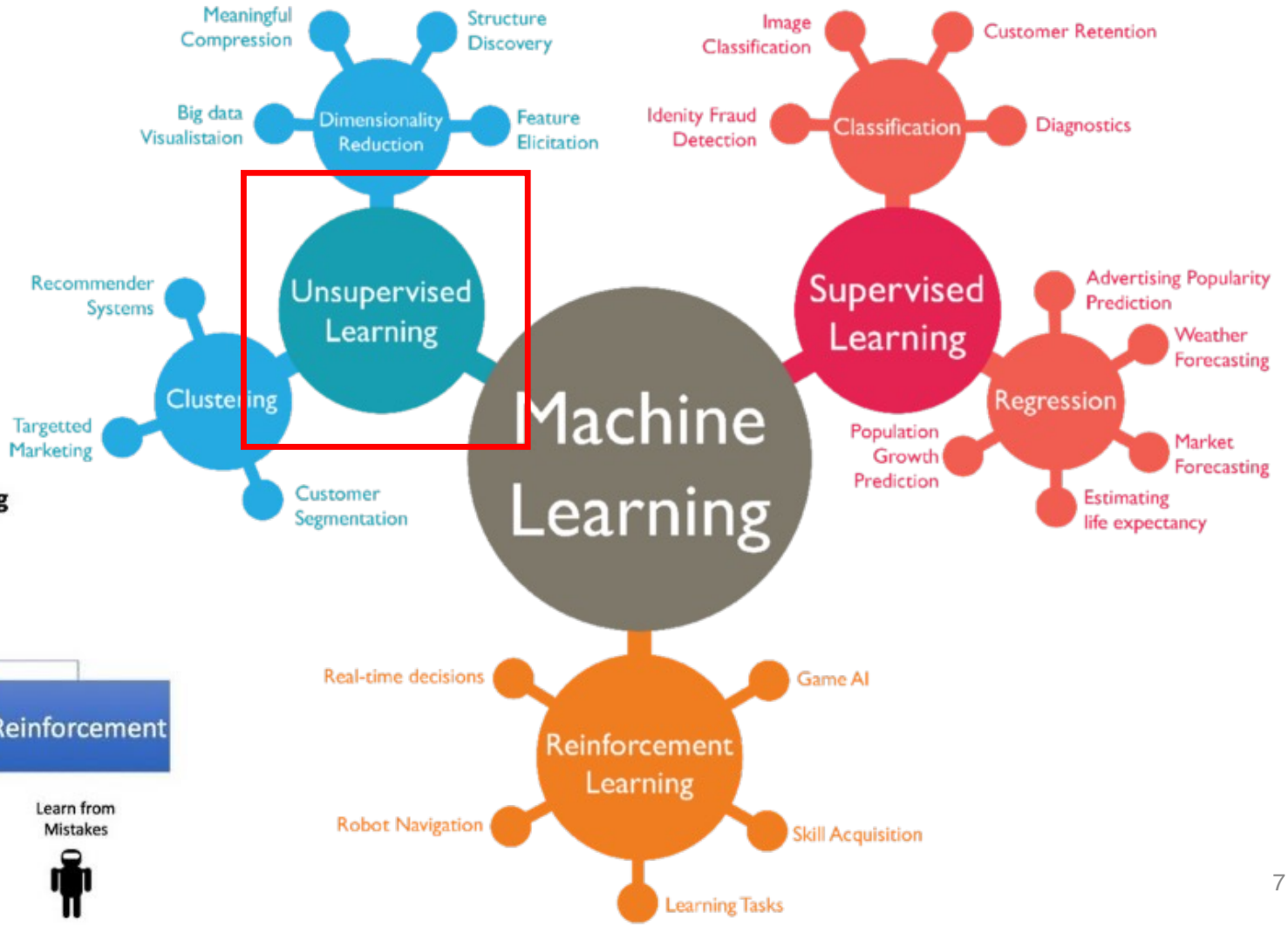
- Target is categorical variable.
- Example
- Covid diagnosis (yes/no)
- Disease diagnosis from gait information:
  - 1) Normal,
  - 2) Sick/Knee OA
  - 3) Sick/Parkinson



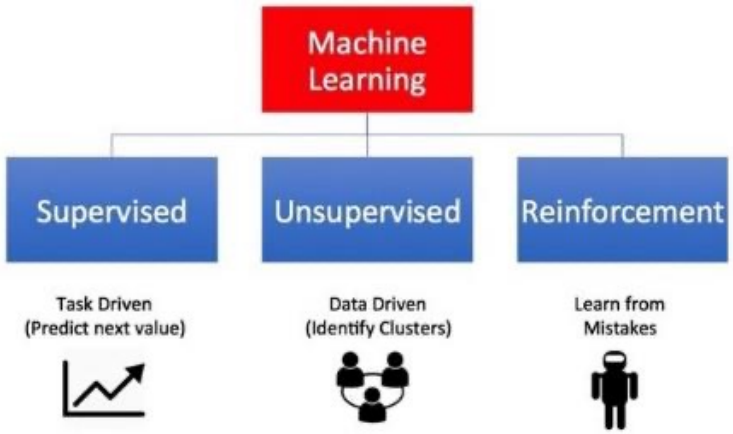
Regression

- Target is numeric variable.
- Example
- PD's state diagnosis from movement data.
- Glucose level prediction from breath particles.

Task2: Unsupervised learning (descriptive task)



Types of Machine Learning



Task2: Unsupervised learning (descriptive task)

Training Data

inputs

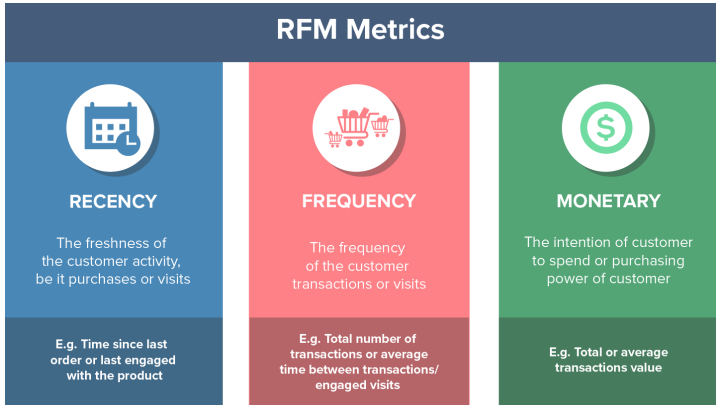
target

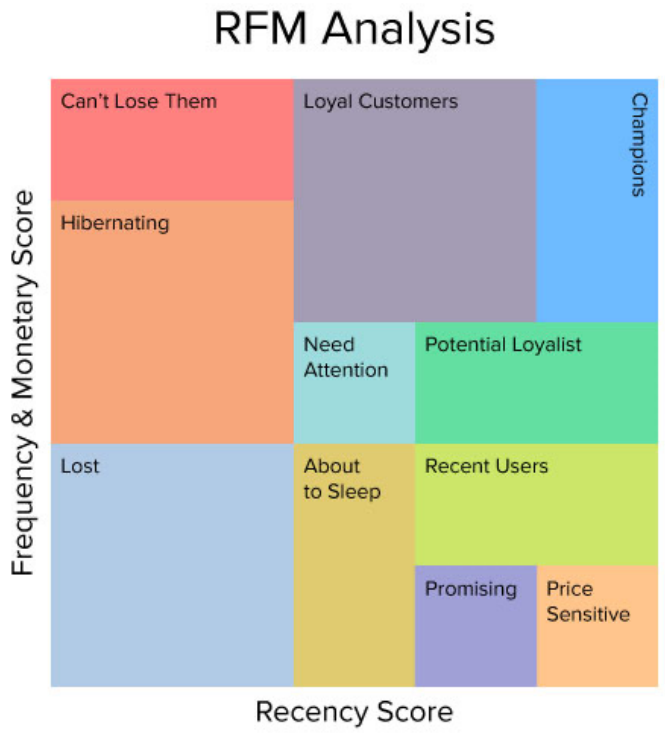
Age	Income	Gender	Province	Purchase
25	25,000	Female	Bangkok	Yes
35	50,000	Female	Nontaburi	Yes
32	35,000	Male	Bangkok	

Testing Data

Age	Income		Province	Purchase
25	25,000		Bangkok	?

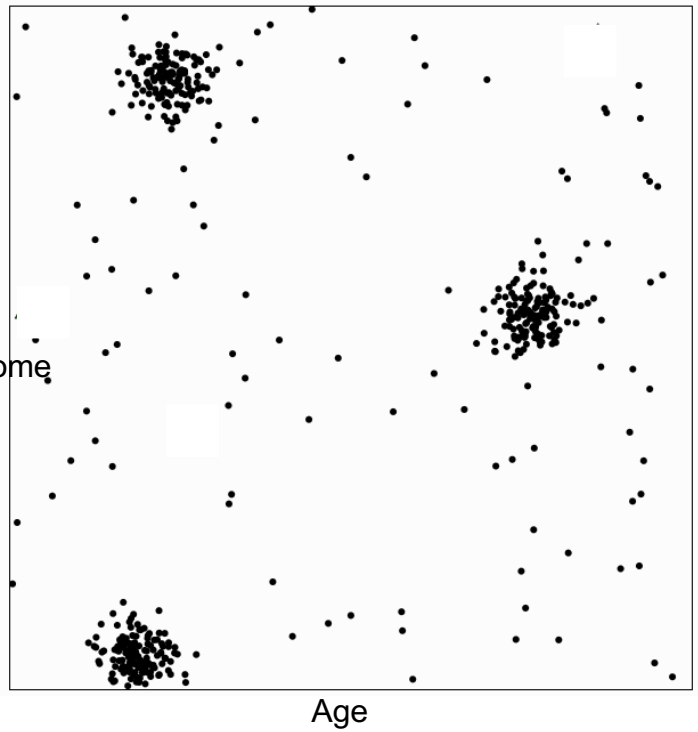
Clustering

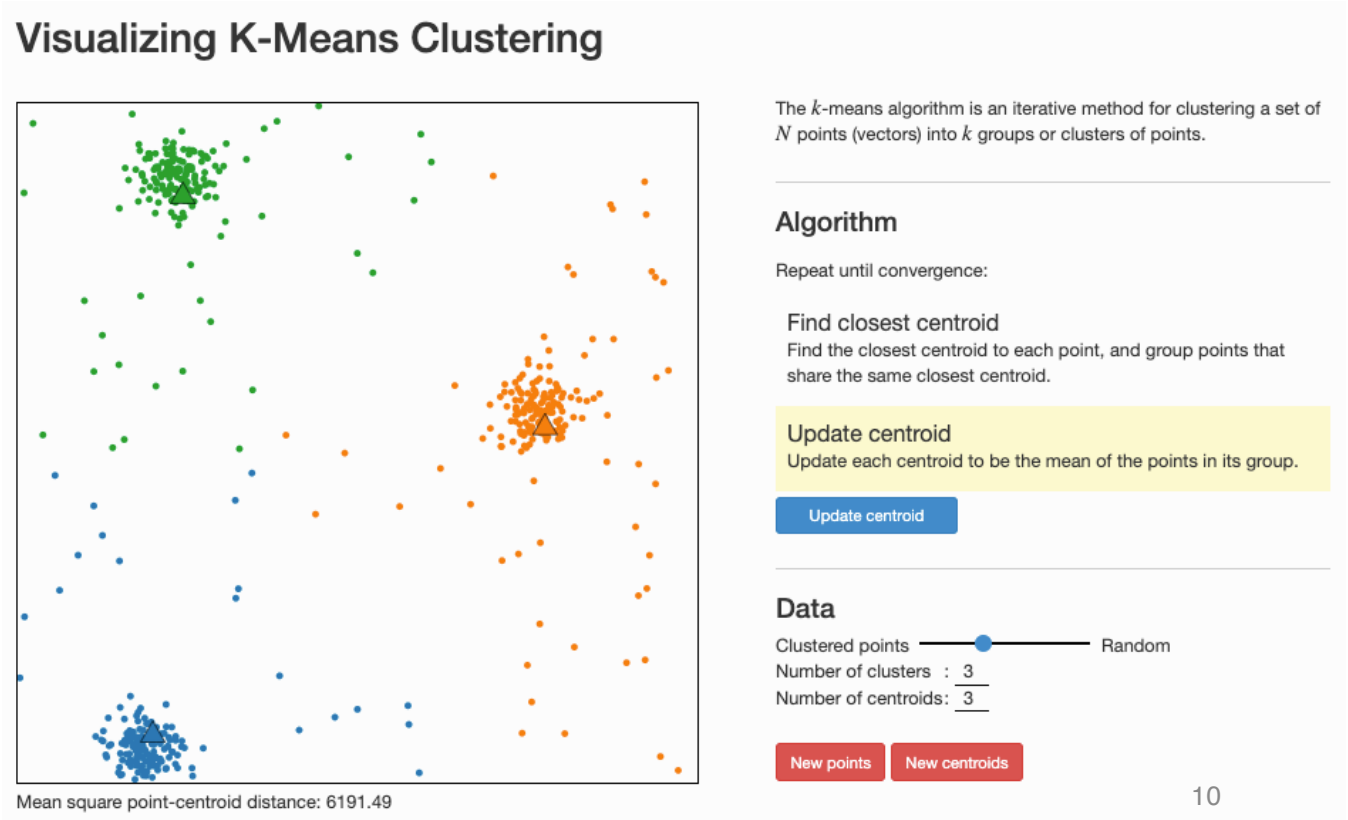

RFM Metrics
RECENCY
The freshness of the customer activity, be it purchases or visits
E.g. Time since last order or last engaged with the product
FREQUENCY
The frequency of the customer transactions or visits
E.g. Total number of transactions or average time between transactions/ engaged visits
MONETARY
The intention of customer to spend or purchasing power of customer
E.g. Total or average transactions value


RFM Analysis
Frequency & Monetary Score
Can't Lose Them
Loyal Customers
Champions
Hibernating
Need Attention
Potential Loyalist
Lost
About to Sleep
Recent Users
Promising
Price Sensitive
Recency Score

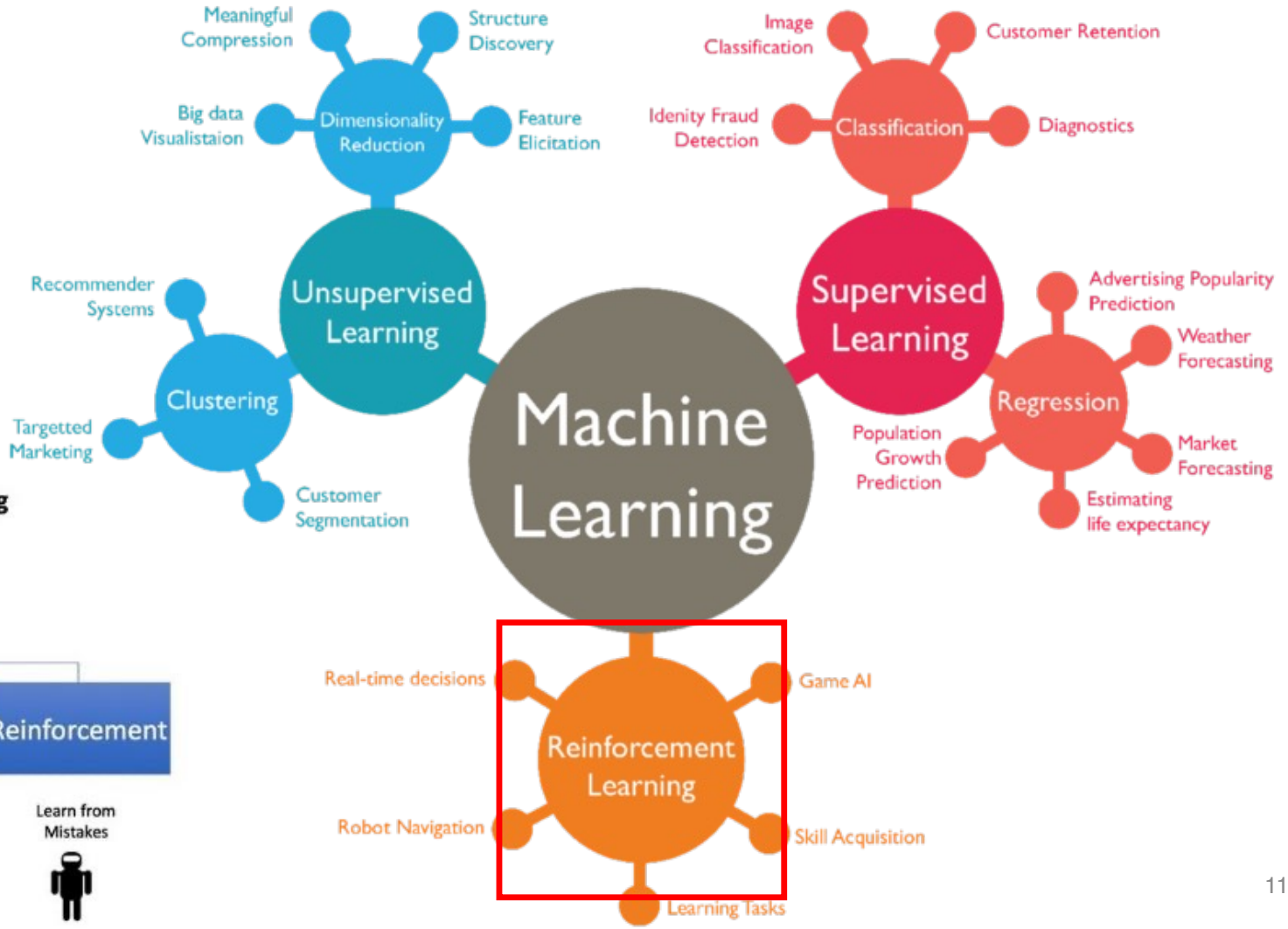
K-means Clustering

- http://web.stanford.edu/class/ee103/visualizations/kmeans/kmeans.html

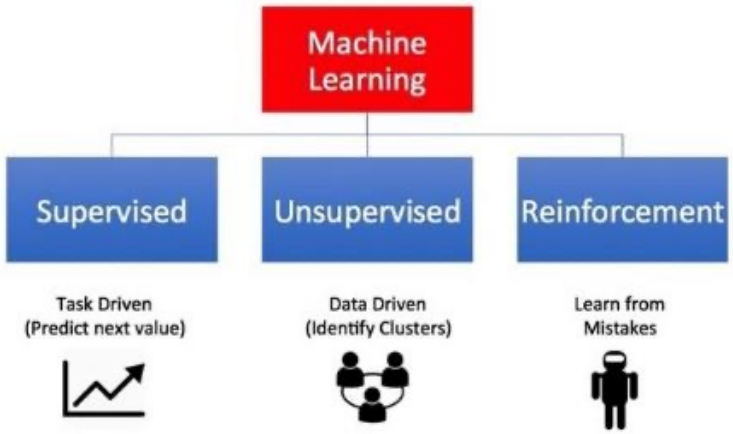




Task3: Reinforcement learning (optimization task)

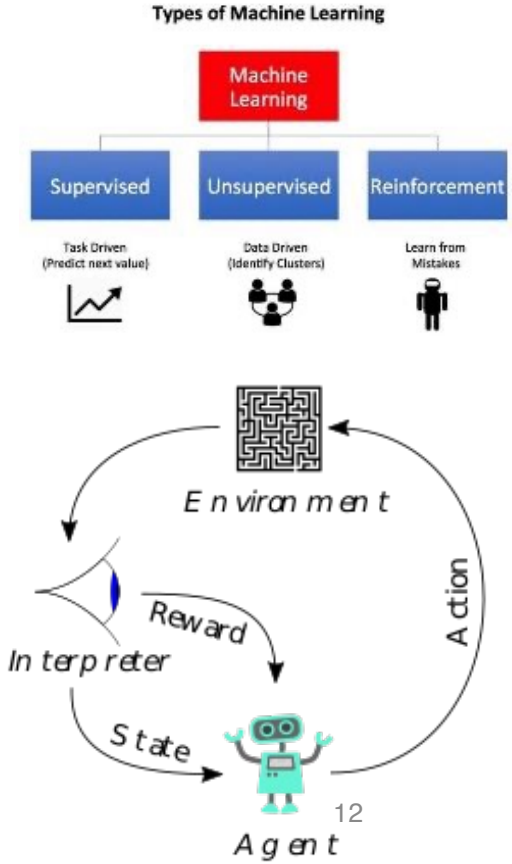


Types of Machine Learning



Task3: Reinforcement Learning (cont.)

https://www.youtube.com/watch?v=fiQsmdwEGT8



https://www.zdnet.com/article/deepmind-alphago-zero-learns-on-its-own-without-meatbag-intervention/

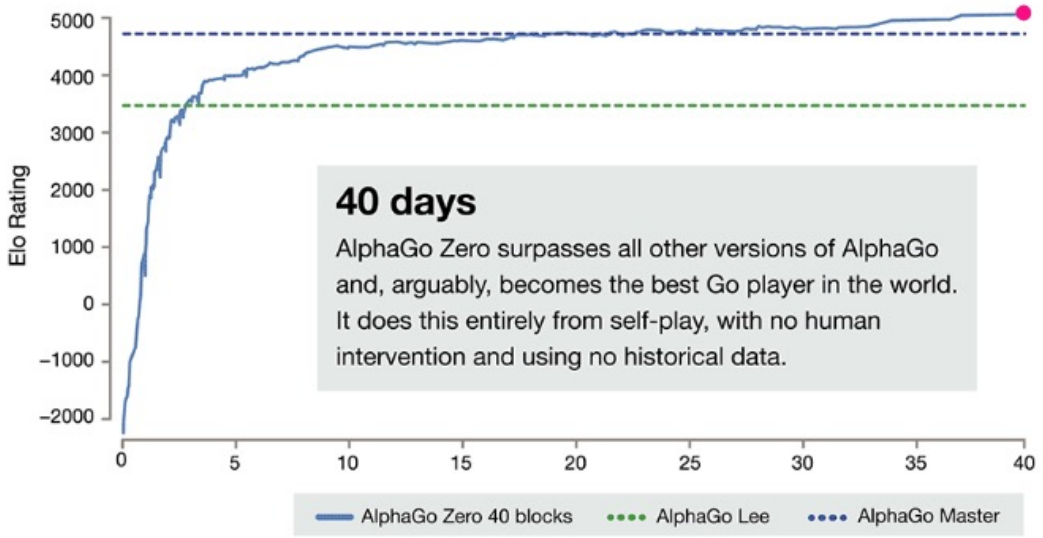
Reinforcement Learning (cont.)

DeepMind AlphaGo Zero learns on its own without meatbag intervention

The latest iteration of DeepMind's Go-playing AI has taught itself and bested other versions of AlphaGo.

By Chris Duckett | October 19, 2017 -- 00:44 GMT (08:44 GMT+08:00) | Topic: Innovation

Is AlphaGo Zero a scientific breakthrough – step towards AGI?



หัวข้อการแข่งขันที่ 3 2019 - 2020

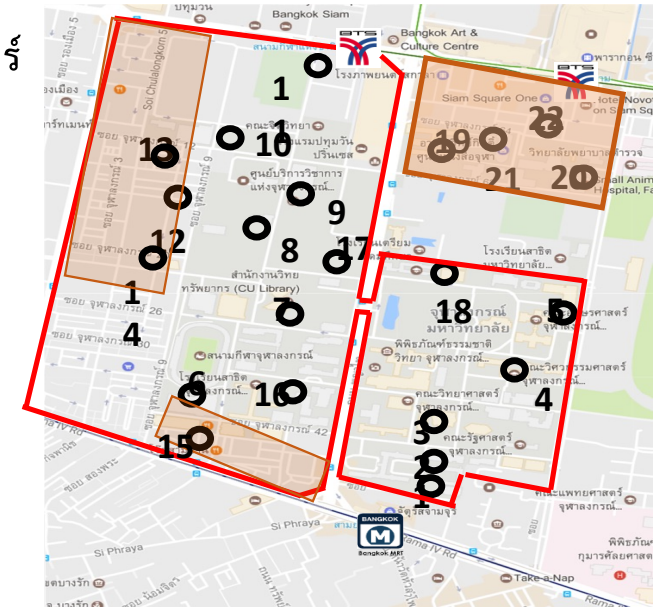
การออกแบบระบบ Relocation สำหรับ CU TOYOTA Ha:mo

CU TOYOTA Ha:mo OPEN INNOVATION CONTEST

พื้นที่การให้บริการ 22 สถานี 60 ช่องจอด

ครอบคลุมบริเวณมหาวิทยาลัย ศูนย์การค้า และ สยามสแควร์

1) ทางออกไปจามจุรีสแควร์	12) ระเบียงจามจุรี
2) เศรษฐศาสตร์	13) สวนหลวงสแควร์
3) ศาลาพระเกี้ยว	14) แอมพาร์ค
4) วิศวกรรมศาสตร์	15) ยู เซ็นเตอร์
5) อักษรศาสตร์	16) นิเทศศาสตร์
6) จามจุรี 9	17) สำนักงานทรัพย์สิน
7) จามจุรี 5	18) อาคารศิลปวัฒนธรรม
8) หอพักวิทยนิเวศน์	19) เภสัชศาสตร์
9) จามจุรี 10	20) สัตวแพทยศาสตร์
10) จุฬาพัฒน์ 14	21) อาคารวิทยกิตติ์
11) บีทีเอส สนามกีฬาแห่งชาติ	22) สยามสแควร์ ซอย 8



<u>เปิดให้บริการ</u> **จันทร์ - ศุกร์ 7.00-19.00**

<u>อัตราค่าบริการ</u>

การใช้บริการ **30 บาท / 20 นาทีแรก เกินจากนั้นนาทีละ 2 บาท**

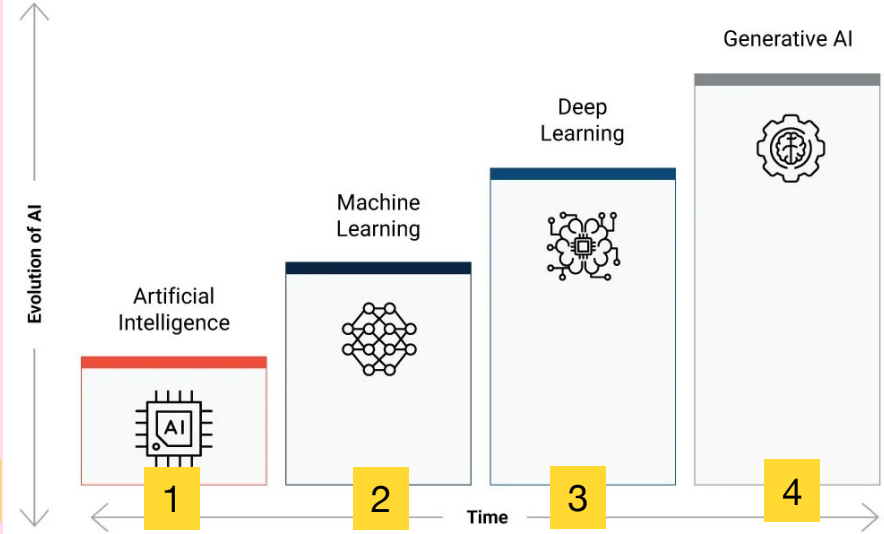
ค่าลงทะเบียนสมาชิก **100 บาท (คืนเป็นคะแนนให้ 100 pt.)**

<u>วิธีชำระค่าบริการ</u> **ผ่านระบบอิเล็คทรอนิก ด้วยบัตรเครดิต/เดบิต**

Generative AI (Gen AI)

Gen AI

Not automatic, not prediction

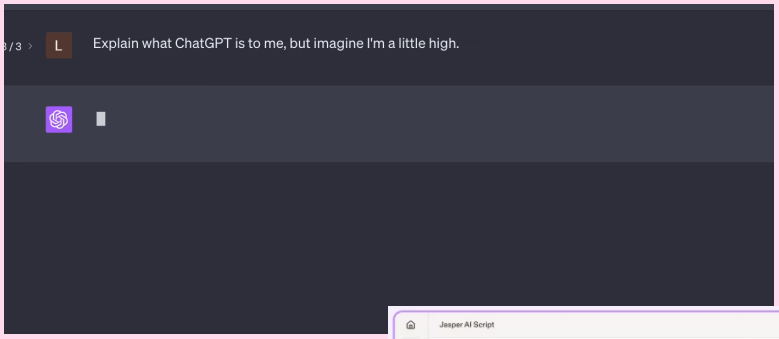


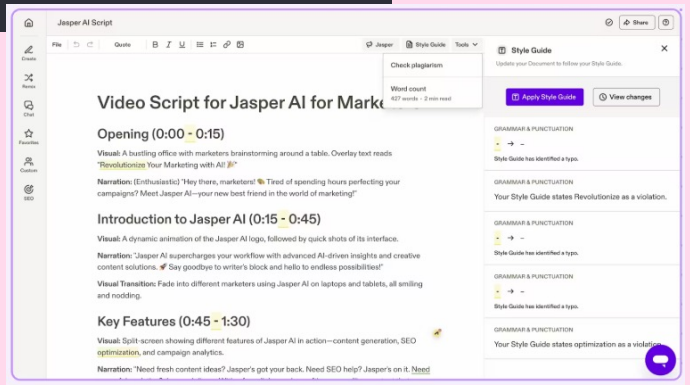
1) Assistant Chatbot
2) Text Summarization
3) Content Creation
4) Data Analytics
5) Etc.

Claude

Gemini

deepseek





What is Generative AI?

Generative artificial intelligence (AI) is a type of AI that can create new content, such as text, images, music, and videos. It uses machine learning models to learn from its training data to “generate” new statistically probable outputs.

Artificial Intelligence

Is the field of study

Machine Learning

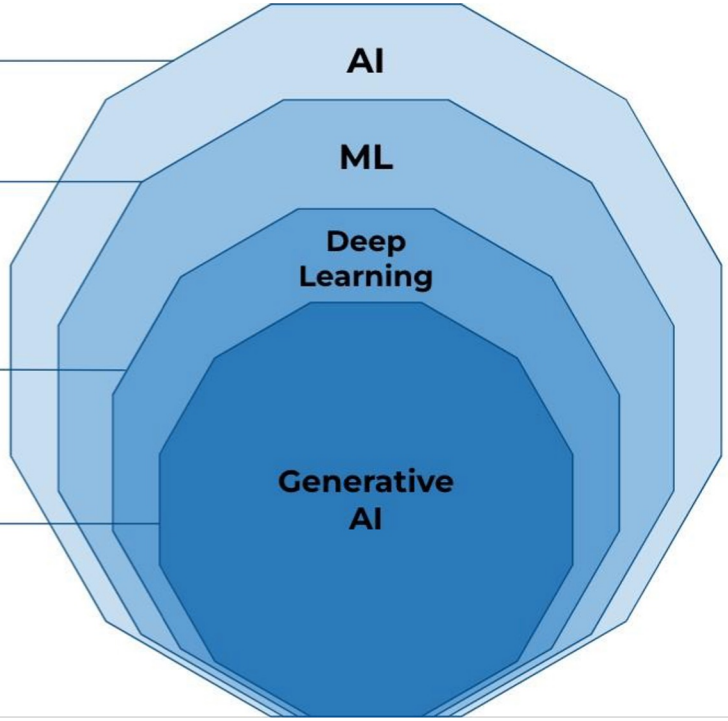
Is a branch of AI that focus on the creation of intelligent machines that learn from data. Another very well know branch inside AI is **Optimization**.

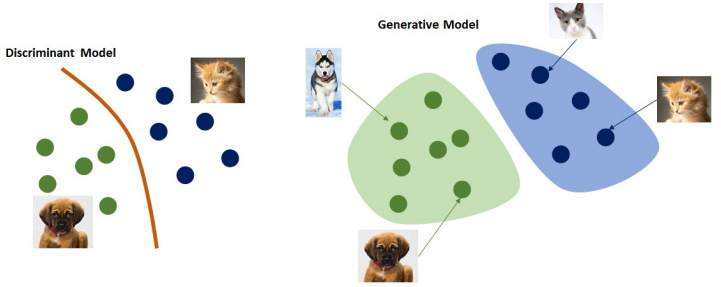
Deep Learning

Is a subset of Machine Learning methods, based on **Artificial Neural Networks.** Examples: CNNs, RNNs

Generative AI

A type of ANNs that generate data that is similar to the data it was trained on. Examples: GANs, LLMs





What can Gen AI create?

- Text
- Image
- Video
- Code
- Multimodal

ChatGPT

- ChatGPT was launched by OpenAI on 30 Nov 2022.
- ChatGPT is a large language model (LLM) for conversational AI applications.
- Generates human-like text and performs NLP tasks.
- Scalable and flexible for various use cases.
- ChatGPT didn't enclose the details.

OpenAI

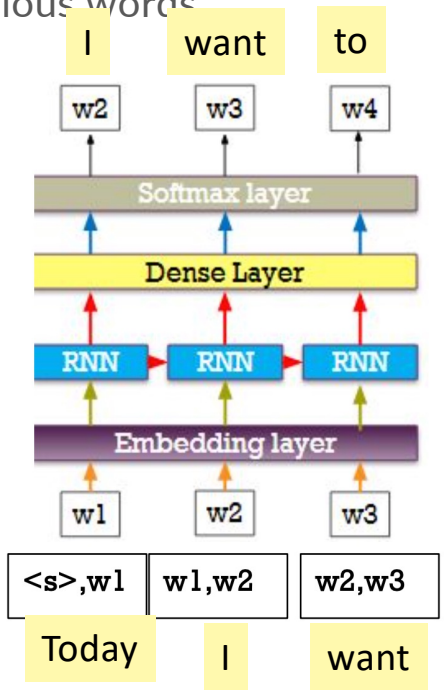
Language Model (LM)

- It is the model that aims to predict next word based on the given previous words.
- So, the model can understand grammar & context.

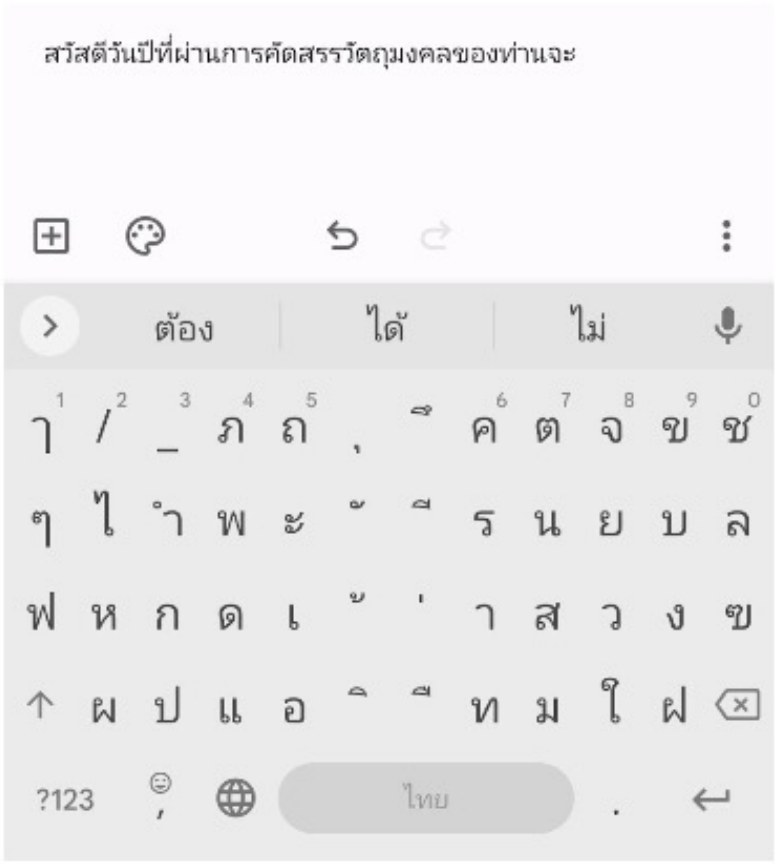
วันนี้เราอยากจะกินข้าวมัน____

Today I want to eat chick ____ .

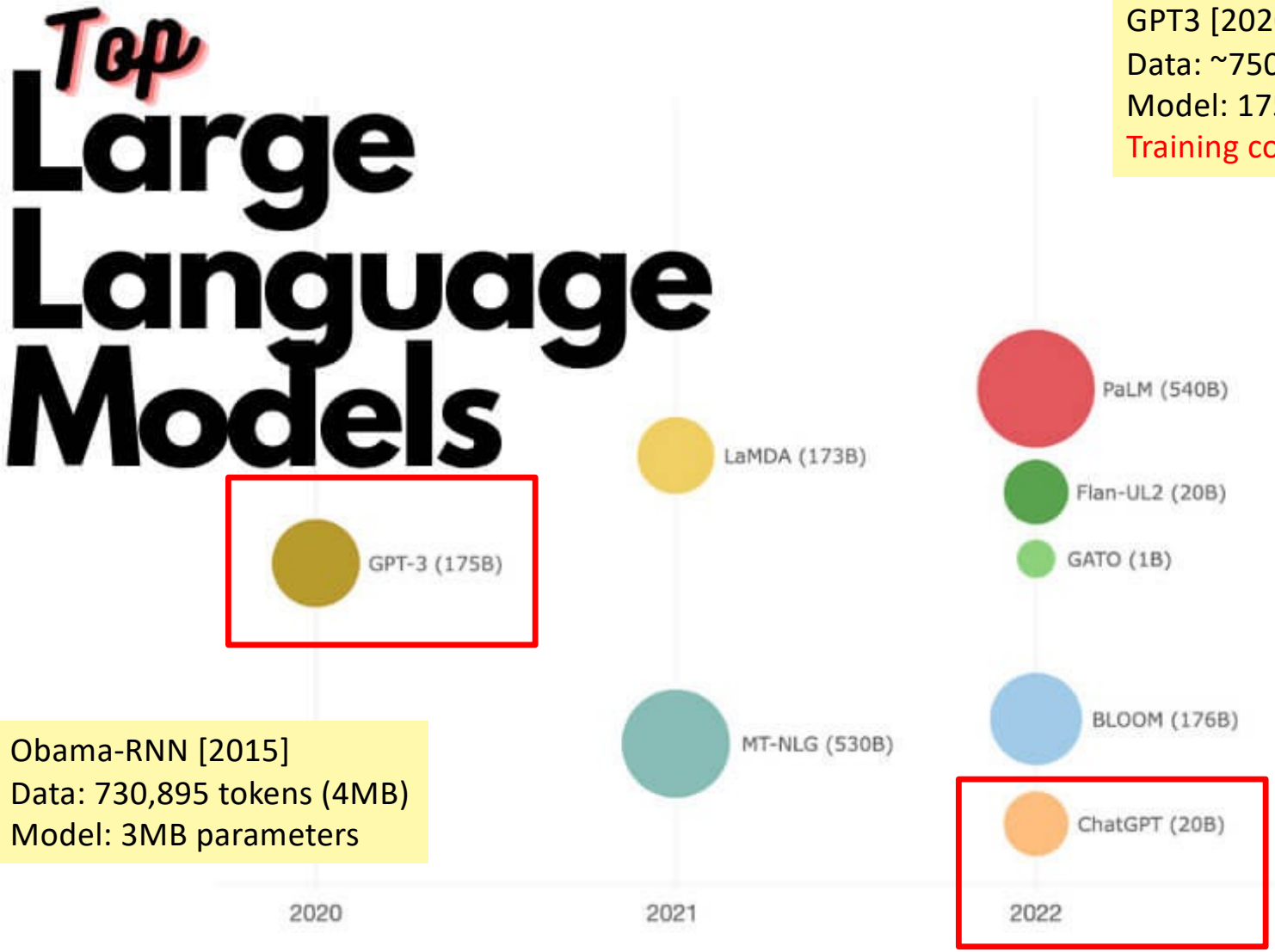
วัน นี้ เรา อยาก จะ กิน ข้าว มัน



You have been using LM ☺ - Autocomplete

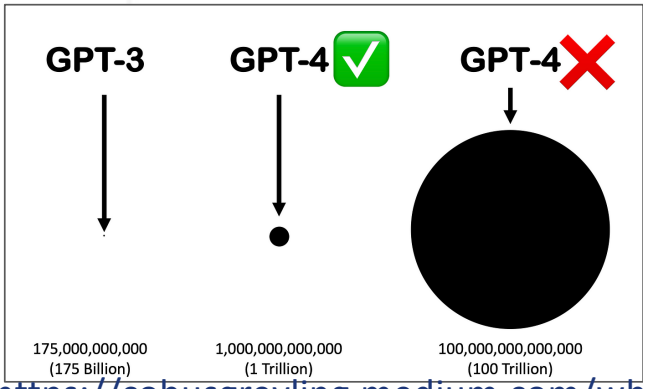


Top Large Language Models



GPT3 [2020]
Data: ~750GB (30,000x)
Model: 175B parameters (700,000x)
Training cost: $5M, equivalent to ~300 years

Obama-RNN [2015]
Data: 730,895 tokens (4MB)
Model: 3MB parameters



https://cobusgreyling.medium.com/what-are-realistic-gpt-4-size-expectations-73f00c39b832

https://vectara.com/top-large-language-models-llms-gpt-4-llama-gato-bloom-and-when-to-choose-one-over-the-other/

vectara

Step 1

Collect demonstration data and train a supervised policy.

LM

A prompt is sampled from our prompt dataset.

Explain reinforcement learning to a 6 year old.

A labeler demonstrates the desired output behavior.

We give treats and punishments to teach...

This data is used to fine-tune GPT-3.5 with supervised learning.

SFT

Step 2

Collect comparison data and train a reward model.

Reward Model (Scoring Model)

A prompt and several model outputs are sampled.

Explain reinforcement learning to a 6 year old.

A In reinforcement learning, the agent is...

B Explain rewards...

C In machine learning...

D We give treats and punishments to teach...

A labeler ranks the outputs from best to worst.

D > C > A > B

This data is used to train our reward model.

RM

D > C > A > B

Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

RL

A new prompt is sampled from the dataset.

Write a story about otters.

The PPO model is initialized from the supervised policy.

PPO

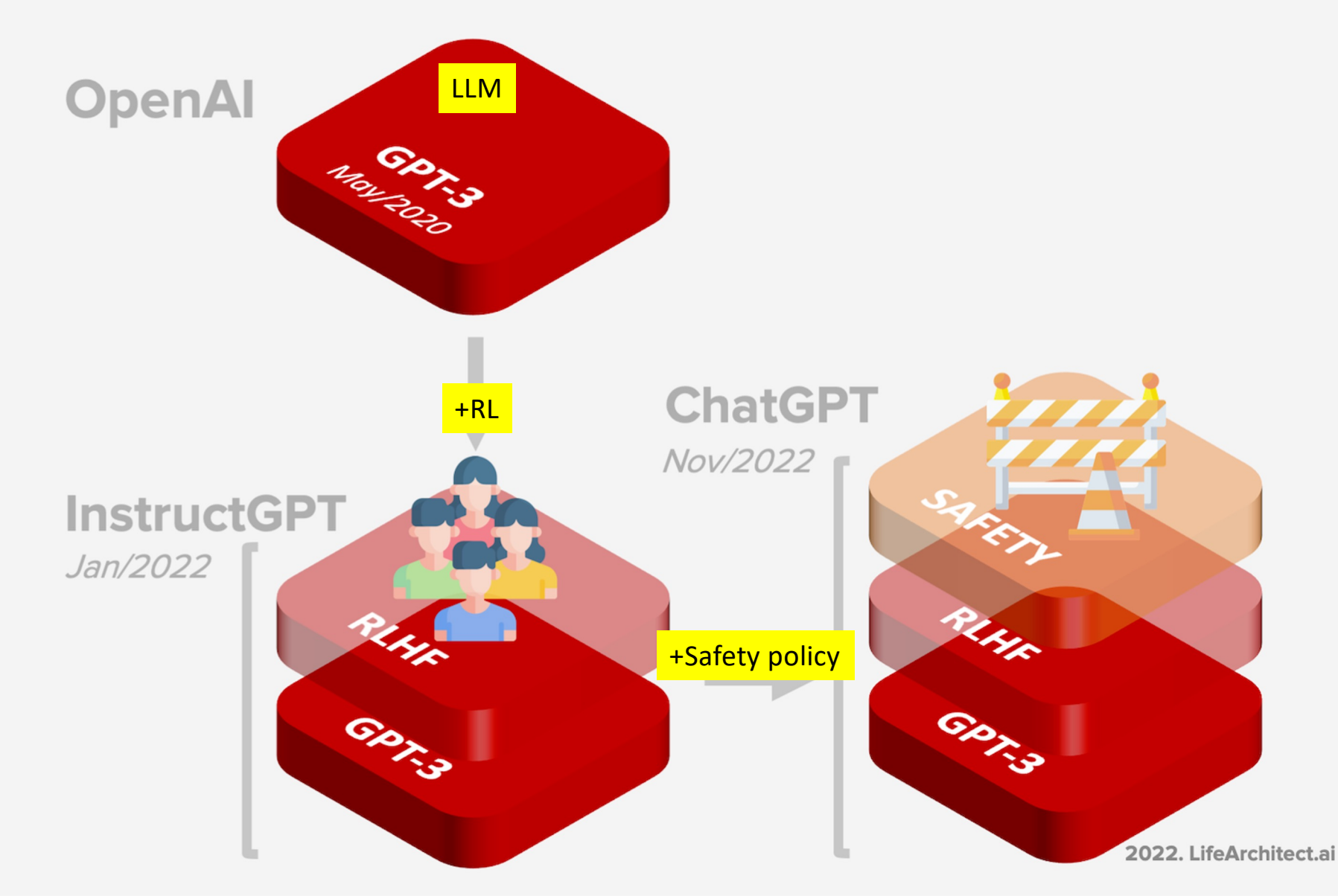
T a

T calculates a reward for the output.

The reward is used to update the policy using PPO.

r_k

https://www.hpc-ai.tech/blog/colossal-ai-chatgpt

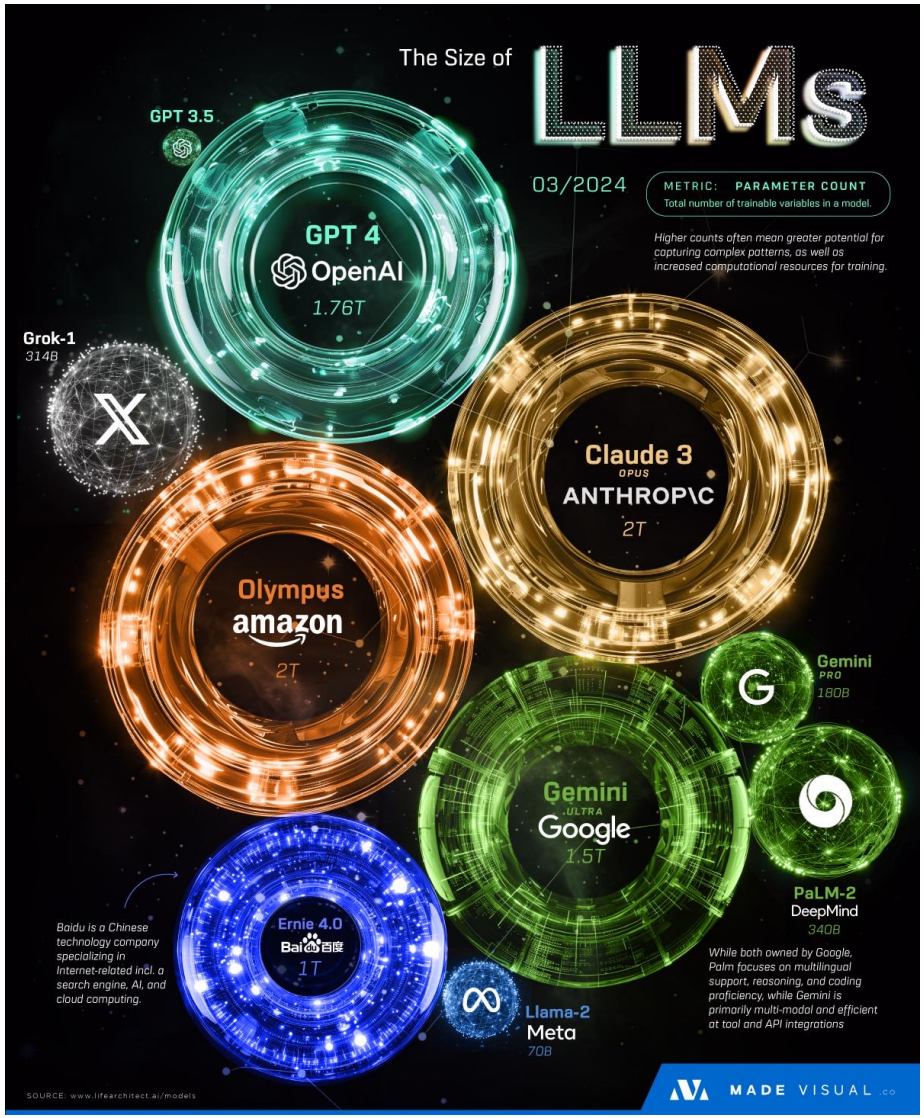

OpenAI
LLM
GPT-3
May/2020
+RL
InstructGPT
Jan/2022
RLHF
GPT-3
+Safety policy
ChatGPT
Nov/2022
SAFETY
RLHF
GPT-3
2022. LifeArchitect.ai

Large + LM = LLM

Model	Year	Parameters (approx)	Tokens Seen (approx)
GPT-1	2018	**117M**	~0.1B
GPT-2	2019	**1.5B**	~10B
GPT-3	2020	**175B**	~300B
GPT-4	2023	**~1.7T** (MoE est.)	~13T
GPT-5	2025	**300B**	~114T

Amazon now develops its own family of **foundation models** called **Amazon Nova** (launched late 2024):

- **Nova Micro** → lightweight text-only model (fast, low cost)
- **Nova Lite** → multimodal (text, images, video), efficiency-focused
- **Nova Pro** → stronger multimodal, balance between accuracy & speed
- **Nova Premier** → top-tier multimodal model (teacher model for distillation)
- **Nova Canvas** → image generation
- **Nova Reel** → video generation
- **Nova Sonic** → speech-to-speech (voice)
- **Nova Act** → agent model that can operate tools and browsers



Popular LLM

Currently, large language models are very popular and there are many models available for use. They are categorized by their application as follows:

1. **Open-source models:** Llama-4, Mistral, DeepSeek-V3, Gemma-3, Qwen-3
2. **Closed-source models:** GPT-5, Claude, Gemini
3. **Specific models:** Typhoon-2.5, SEA-LION

https://wandb.ai/darek/llmapps/reports/A-Gentle-Introduction-to-LLM-APIs--Vmlldzo0NjM0MTMz

Large Language Model Application

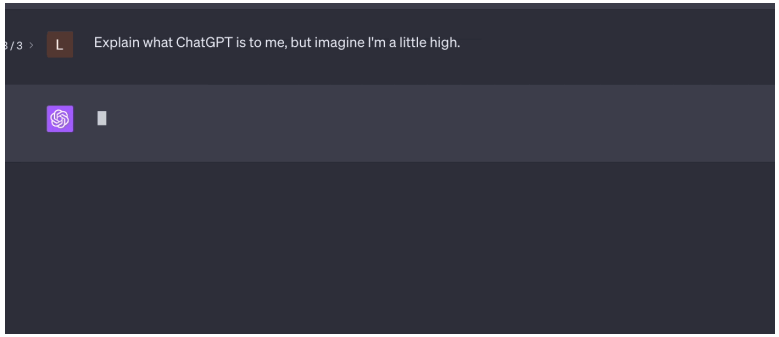
LLM has been applied in various fields related to language

1. **Assistant Chatbot**
2. **Text Summarization**
3. **Content Creation**
4. **Data Analytics**

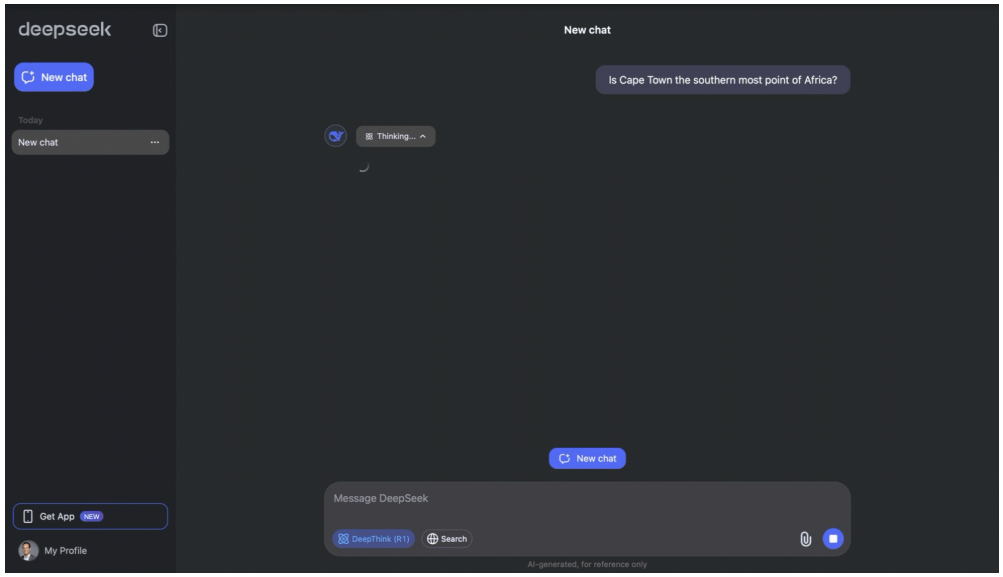
Assistant Chatbot

Mostly, the use of Large Language Models to answer questions, explain information, or assist with various tasks received from users in a natural way is the most popular, with many products available for use, such as ChatGPT, Claude, and DeepSeek Chat.

ChatGPT



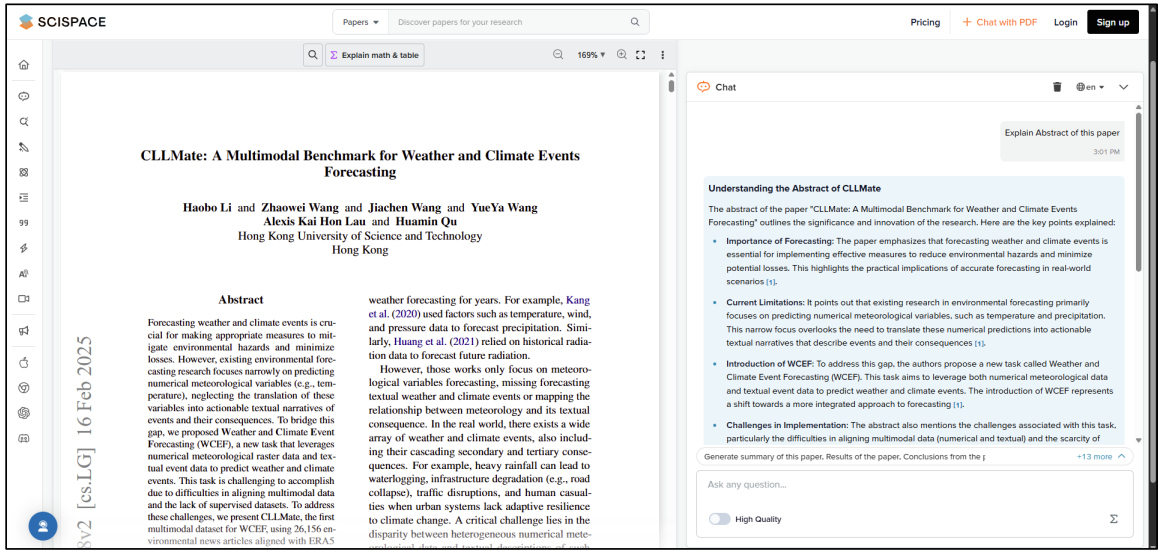
deepseek



https://chatgpt.com/ , https://chat.deepseek.com/

Text Summarization

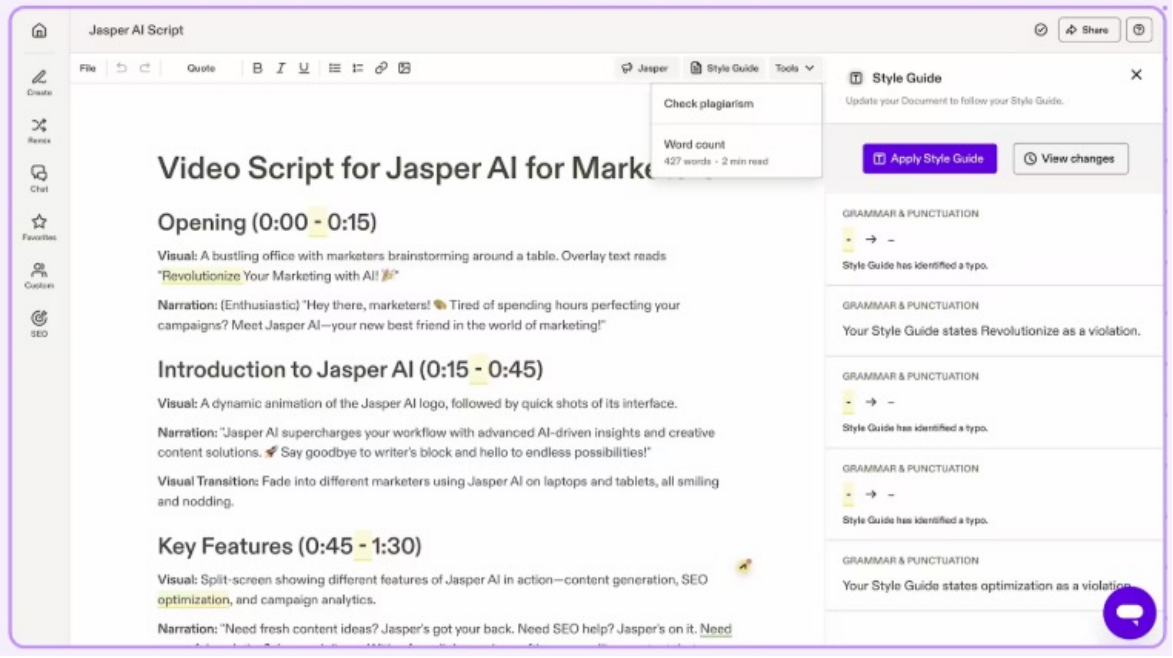
We can integrate Large Language Models with document data such as books and research papers to enhance the model's knowledge base, enabling it to answer questions based on the information contained in the documents. For example, SciSpace is an application that allows users to ask questions and summarize information from research papers they input.



https://scispace.com/

Content Creation

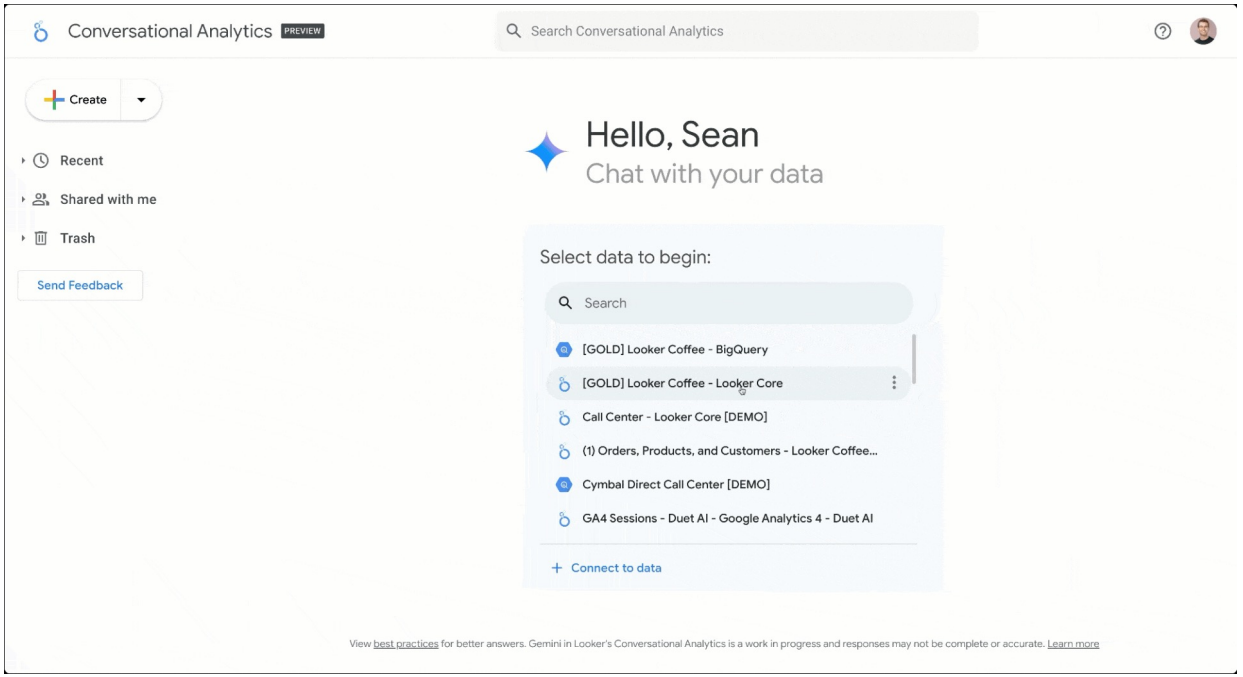
Large Language Models can also be used to write articles, design advertisements, write programs, or create creative content, such as Jasper AI for marketing article writing.



https://www.jasper.ai/

Data Analytics

Large Language Models are also used for data analysis and finding insights within data, such as Looker Conversational Analytics for asking questions about the database and displaying graphs from those questions.



https://cloud.google.com/blog/products/business-intelligence/conversational-analytics-in-looker-is-now-in-preview

LLM Techniques

LLM Techniques - Overview

To improve model performance, three common approaches are generally used, ranked from **easiest to most difficult** to implement:

1. **Prompting** is a technique for designing commands or questions to be as intelligent and clear as possible, so that the model understands the context and responds more accurately.

2. **Retrieval-Augmented Generation (RAG)** is a technique that helps models "understand context better" by connecting external information such as documents, databases, or websites. Before answering a question, the model retrieves related information and then uses that information as context to generate reliable answers that can be referenced to their sources.

3. **Fine-tuning** is the process of retraining a pre-trained LLM model with our specific data, such as customer conversation texts, industry-specific language usage, or new problems that still require the same knowledge base. This helps the model understand the specific context and respond more accurately than general models.

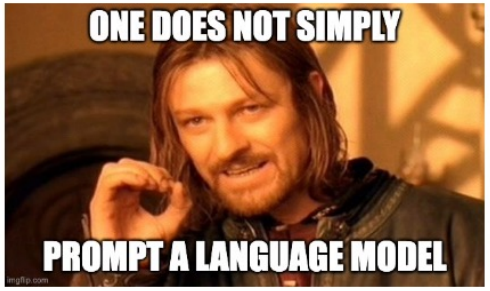
4. **Agentic AI:** Enabling the model to reason step-by-step, use memory, and call tools autonomously to solve complex, multi-stage tasks.

1. Prompting

Prompt engineer aims to solve alignment problem.

- Language Model (LM) is originally trained to predict the next word, NOT answer the question.
- GPT (GPT3 is 175B parameters) is usually frozen (not trained).
- Since we cannot change the model, we need to align (change) the question (also called prompt).

Input (Prompt)	Output
The patient was died.	The patient's body was found in a dark alley behind the hospital's…
"The patient was died." correct this	claim if you really believe such figures….
Poor English input: The patient was died.	Good English output: The patient died.

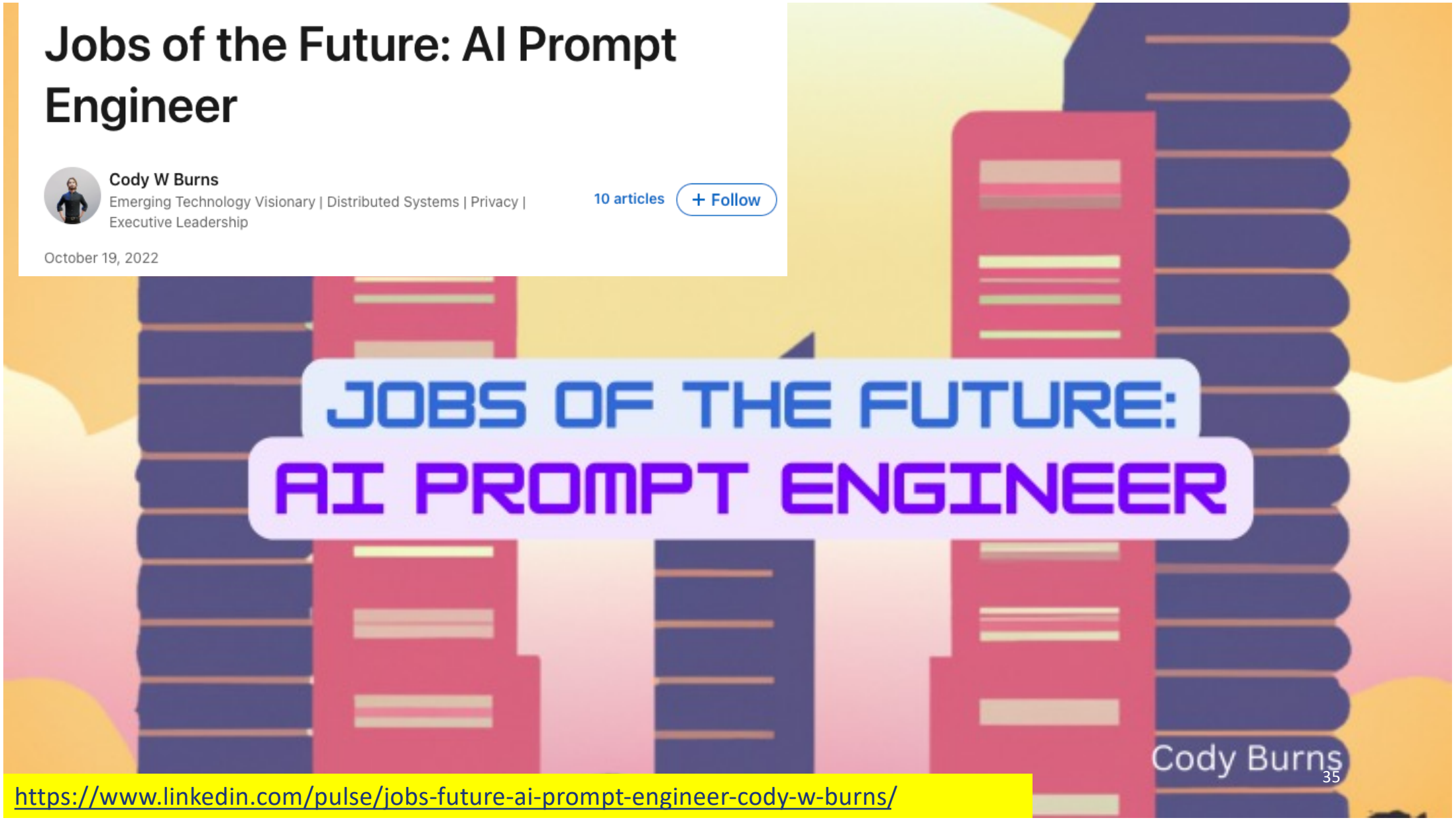


Jobs of the Future: AI Prompt Engineer

Cody W Burns
Emerging Technology Visionary | Distributed Systems | Privacy | Executive Leadership

10 articles + Follow

October 19, 2022



https://www.linkedin.com/pulse/jobs-future-ai-prompt-engineer-cody-w-burns/

Prompt Engineering Techniques

servicenow

https://www.servicenow.com/ai/what-is-prompt-engineering.html

Prompt types

1) User prompt: the specific instructions or questions a user provides to an AI system to elicit a desired response.

2) System prompt: the foundational instructions that dictate an AI's behavior.

```
const response = await openai.chat.completions.create({
  model: "gpt-4o",
  messages: [
    {
      "role": "developer",
      "content": [
        {
          "type": "text",
          "text": `
            You are a helpful assistant that answers programming
            questions in the style of a southern belle from the
            southeast United States.
          `
        }
      ]
    },
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "Are semicolons optional in JavaScript?"
        }
      ]
    }
  ],
  store: true,
});
```

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=2048,
    system="You are a seasoned data scientist at a Fortune 500 company.", # <-- rol
    messages=[
        {"role": "user", "content": "Analyze this dataset for anomalies: <dataset>{
    ]
)

print(response.content)
```

Prompting Principles

The principles for crafting good prompts to get more accurate answers are twofold:

Principle 1: Write clear and specific instructions

Principle 2: Give the model time to “think”

Tactic 1.1: Delimiters (“”, :, etc.) to indicate parts of the input

Task: Summarize text

- Delimiters can be anything like: ```, """, < >, `<tag> </tag>`, `:`

In [3]:
```
text = f"""
You should express what you want a model to do by \
providing instructions that are as clear and \
specific as you can possibly make them. \
This will guide the model towards the desired output, \
and reduce the chances of receiving irrelevant \
or incorrect responses. Don't confuse writing a \
clear prompt with writing a short prompt. \
In many cases, longer prompts provide more clarity \
and context for the model, which can lead to \
more detailed and relevant outputs.
"""
prompt = f"""
Summarize the text delimited by triple backticks \
into a single sentence.
```{text}```
"""
response = get_completion(prompt)
print(response)
```

```
Providing clear and specific instructions to a model is essential for gui
ding it towards the desired output and reducing the chances of irrelevant
or incorrect responses, with longer prompts often providing more clarity
and context for more detailed and relevant outputs.
```
```

Tactic 1.2: Structured output (and also input)

Task: Generate structured outputs

In [4]:
```
prompt = f"""
Generate a list of three made-up book titles along \
with their authors and genres.
Provide them in JSON format with the following keys:
book_id, title, author, genre.
"""
response = get_completion(prompt)
print(response)
```

```
[
    {
        "book_id": 1,
        "title": "The Midnight Garden",
        "author": "Elena Rivers",
        "genre": "Fantasy"
    },
    {
        "book_id": 2,
        "title": "Echoes of the Past",
        "author": "Nathan Black",
        "genre": "Mystery"
    },
    {
        "book_id": 3,
        "title": "Whispers in the Wind",
        "author": "Samantha Reed",
        "genre": "Romance"
    }
]
```

Tactic 1.3: Check for conditions

Task: Convert paragraph to line-by-line steps

In [5]:
```
text_1 = f"""
Making a cup of tea is easy! First, you need to get some \
water boiling. While that's happening, \
grab a cup and put a tea bag in it. Once the water is \
hot enough, just pour it over the tea bag. \
Let it sit for a bit so the tea can steep. After a \
few minutes, take out the tea bag. If you \
like, you can add some sugar or milk to taste. \
And that's it! You've got yourself a delicious \
cup of tea to enjoy.
"""

prompt = f"""
You will be provided with text delimited by triple quotes.
If it contains a sequence of instructions, \
re-write those instructions in the following format:

Step 1 - ...
Step 2 - …
…
Step N - …

If the text does not contain a sequence of instructions, \
then simply write \"No steps provided.\"

\"\"\"{text_1}\"\"\"
"""
response = get_completion(prompt)
print("Completion for Text 1:")
print(response)
```

```
Completion for Text 1:
Step 1 - Get some water boiling.
Step 2 - Grab a cup and put a tea bag in it.
Step 3 - Pour the hot water over the tea bag.
Step 4 - Let the tea steep for a few minutes.
Step 5 - Remove the tea bag.
Step 6 - Add sugar or milk to taste.
Step 7 - Enjoy your delicious cup of tea.
```

In [6]:
```
text_2 = f"""
The sun is shining brightly today, and the birds are \
singing. It's a beautiful day to go for a \
walk in the park. The flowers are blooming, and the \
trees are swaying gently in the breeze. People \
are out and about, enjoying the lovely weather. \
Some are having picnics, while others are playing \
games or simply relaxing on the grass. It's a \
perfect day to spend time outdoors and appreciate the \
beauty of nature.
"""
prompt = f"""
You will be provided with text delimited by triple quotes.
If it contains a sequence of instructions, \
re-write those instructions in the following format:

Step 1 - ...
Step 2 - …
…
Step N - …

If the text does not contain a sequence of instructions, \
then simply write \"No steps provided.\"

\"\"\"{text_2}\"\"\"
"""
response = get_completion(prompt)
print("Completion for Text 2:")
print(response)
```

```
Completion for Text 2:
No steps provided.
```

Tactic 1.4: Few-shot prompting (give an example)

Task: Answer in a consistent style

```
In [7]: prompt = f"""
        Your task is to answer in a consistent style.

        <child>: Teach me about patience.

        <grandparent>: The river that carves the deepest \
        valley flows from a modest spring; the \
        grandest symphony originates from a single note; \
        the most intricate tapestry begins with a solitary thread.

        <child>: Teach me about resilience.
        """
        response = get_completion(prompt)
        print(response)
```

Few-shot prompting

```
<grandparent>: The tallest trees weather the strongest storms; the bright
est stars shine in the darkest nights; the strongest hearts are forged in
the hottest fires.
```

Prompting Principles

The principles for crafting good prompts to get more accurate answers are twofold:

Principle 1: Write clear and specific instructions

Principle 2: Give the model time to “think”

Tactic 2.1: Lay out the steps to complete the task (clear instructions)

In [8]:
```
text = f"""
In a charming village, siblings Jack and Jill set out on \
a quest to fetch water from a hilltop \
well. As they climbed, singing joyfully, misfortune \
struck—Jack tripped on a stone and tumbled \
down the hill, with Jill following suit. \
Though slightly battered, the pair returned home to \
comforting embraces. Despite the mishap, \
their adventurous spirits remained undimmed, and they \
continued exploring with delight.
"""
# example 1
prompt_1 = f"""
Perform the following actions:
1 - Summarize the following text delimited by triple \
backticks with 1 sentence.
2 - Translate the summary into French.
3 - List each name in the French summary.
4 - Output a json object that contains the following \
keys: french_summary, num_names.

Separate your answers with line breaks.

Text:
```{text}```
"""
response = get_completion(prompt_1)
print("Completion for prompt 1:")
print(response)
```

```
Completion for prompt 1:
1 - Jack and Jill, siblings from a charming village, go on a quest to fet
ch water from a hilltop well, but encounter misfortune along the way.

2 - Jack et Jill, frère et sœur d'un charmant village, partent en quête d
'eau d'un puits au sommet d'une colline, mais rencontrent des malheurs en
chemin.

3 - Jack, Jill

4 -
{
  "french_summary": "Jack et Jill, frère et sœur d'un charmant village, p
artent en quête d'eau d'un puits au sommet d'une colline, mais rencontren
t des malheurs en chemin.",
  "num_names": 2
}
```

- Explicit steps vs. CoT
- Action1 → Action2 → …
- Reason1 → Reason2 → …

Key Differences

Feature	Explicit Steps in Prompt	Chain-of-Thought (CoT) Prompting
Purpose	Guide step-by-step execution	Encourage **step-by-step reasoning**
Structure	Direct instructions	Model generates intermediate thoughts
Use Case	Coding, workflows, automation	Math, logic, problem-solving
Example	"Step 1: Do X, Step 2: Do Y"	"Let's think step by step..."
```

Tactic 2.2: Let the model work out its own solution first

Chain-of-Thoughts (CoT): Let's model think step by step

Without "thinking"

Answer incorrectly as "True"

```
In [10]: prompt = f"""
Determine if the student's solution is correct or not.

Question:
I'm building a solar power installation and I need \
 help working out the financials.
- Land costs $100 / square foot
- I can buy solar panels for $250 / square foot
- I negotiated a contract for maintenance that will cost \
me a flat $100k per year, and an additional $10 / square \
foot
What is the total cost for the first year of operations
as a function of the number of square feet.

Student's Solution:
Let x be the size of the installation in square feet.
Costs:
1. Land cost: 100x
2. Solar panel cost: 250x
3. Maintenance cost: 100,000 + 100x
Total cost: 100x + 250x + 100,000 + 100x = 450x + 100,000
"""
response = get_completion(prompt)
print(response)
```

```
The student's solution is correct. The total cost for the first year of o
perations as a function of the number of square feet is indeed 450x + 10
0,000.
```

With "thinking"

Answer correctly as "False"

Note that the student's solution is actually not correct.

We can fix this by instructing the model to work out its own solution first.

```
In [11]: prompt = f"""
Your task is to determine if the student's solution \
is correct or not.
To solve the problem do the following:
- First, work out your own solution to the problem includin
- Then compare your solution to the student's solution \
and evaluate if the student's solution is correct or not.
Don't decide if the student's solution is correct until
you have done the problem yourself.

Use the following format:
Question:
```
question here
```
Student's solution:
```
student's solution here
```
Actual solution:
```
steps to work out the solution and your solution here
```
Is the student's solution the same as actual solution \
just calculated:
```
yes or no
```
Student grade:
```
correct or incorrect
```
```

```
Question:
```
I'm building a solar power installation and I need help \
working out the financials.
- Land costs $100 / square foot
- I can buy solar panels for $250 / square foot
- I negotiated a contract for maintenance that will cost \
me a flat $100k per year, and an additional $10 / square \
foot
What is the total cost for the first year of operations \
as a function of the number of square feet.
```
Student's solution:
```
Let x be the size of the installation in square feet.
Costs:
1. Land cost: 100x
2. Solar panel cost: 250x
3. Maintenance cost: 100,000 + 100x
Total cost: 100x + 250x + 100,000 + 100x = 450x + 100,000
```
Actual solution:
"""
response = get_completion(prompt)
print(response)
```

```
Let x be the size of the installation in square feet.

Costs:
1. Land cost: 100x
2. Solar panel cost: 250x
3. Maintenance cost: 100,000 + 10x

Total cost: 100x + 250x + 100,000 + 10x = 360x + 100,000

Is the student's solution the same as actual solution just
calculated:
No

Student grade:
Incorrect
```

2. RAG

Hallucination issue → LLM cannot access a private data!

ChatGPT

พีรพล เวทีกูล คือใคร

พีรพล เวทีกูล เป็นนักข่าวที่มีชื่อเสียงในประเทศไทย โดยเฉพาะในวงการสื่อสารมวลชน เขาเป็นที่รู้จักในฐานะผู้ประกาศข่าวทางช่อง 3 และเป็นพิธีกรรายการที่มีความนิยมในหลายรายการโทรทัศน์ เช่น รายการข่าวพิเศษต่าง ๆ และรายการสารคดีต่าง ๆ ที่ออกอากาศในช่อง 3 ของประเทศไทย

ณรงค์เดช กีรติพรานนท์ คือใคร

ณรงค์เดช กีรติพรานนท์ เป็นนักการเมืองและนักธุรกิจชาวไทยที่มีชื่อเสียง โดยเป็นสมาชิกพรรคการเมืองชั้นนำในประเทศไทย นอกจากนี้เขายังเป็นผู้บริหารในธุรกิจต่าง ๆ

2. Retrieval-Augmented Generation (RAG)

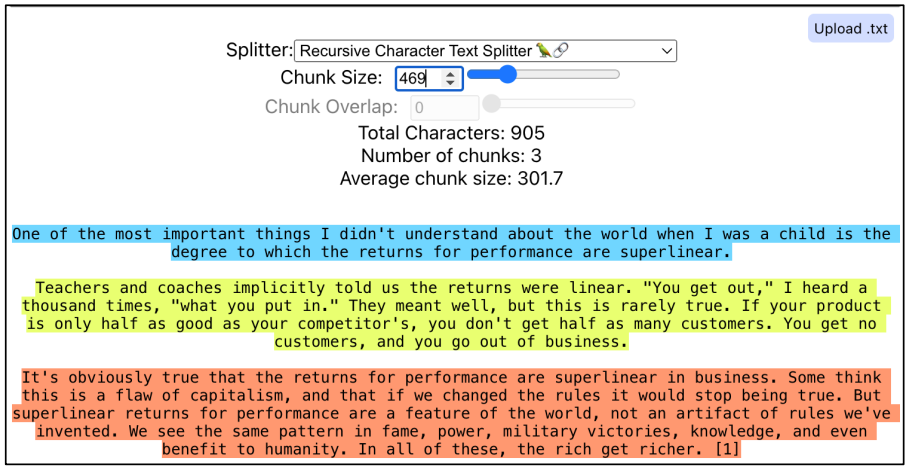

```
LawSections = f"""
มาตรา 1347 เจ้าของที่ดินอาจตัดรากไม้ซึ่งรุกเข้ามาจากที่ดินติดต่อและเอาไว้เสีย ถ้ากิ่งไม้ยื่นล้ำเข้ามา เมื่อเจ้าของที่ดินได้บอกผู้ครอบครองที่ดินติดต่อให้ตัดภายในเวลาอันสมควรแล้ว แต่ผู้นั้นไม่ตัด ท่านว่าเจ้าของที่ดินตัดเอาเสียก็ได้มาตรา 1348 ดอกผลแห่งต้นไม้ที่หล่นตามธรรมดาลงในที่ดินติดต่อ แปลงใด ท่านให้สันนิษฐานไว้ก่อนว่าเป็นดอกผลของที่ดินแปลงนั้น

มาตรา 1349 ที่ดินแปลงใดมีที่ดินแปลงอื่นล้อมอยู่จนไม่มีทางออกถึงทางสาธารณะได้ใช้ ท่านว่าเจ้าของที่ดินแปลงนั้นจะผ่านที่ดินซึ่งล้อมอยู่ไปสู่ทางสาธารณะได้

มาตรา 1350 ถ้าที่ดินแบ่งแยกหรือแบ่งโอนกันเป็นเหตุให้แปลงหนึ่งไม่ มีทางออกไปสู่ทางสาธารณะใช้ ท่านว่าเจ้าของที่ดินแปลงนั้นมีสิทธิเรียกร้อง เอาทางเดินตาม มาตรา ก่อนได้เฉพาะบนที่ดินแปลงที่ได้แบ่งแยกหรือแบ่งโอน กันและไม่ต้องเสียค่าทดแทน

มาตรา 1351 เจ้าของที่ดิน เมื่อบอกล่วงหน้าตามสมควรแล้ว อาจใช้ที่ดินติดต่อเพียงที่จำเป็นในการปลูกสร้างหรือซ่อมแซมรั้ว กำแพง หรือโรงเรือน ตรงหรือใกล้แนวเขตของตน แต่จะเข้าไปในเรือนที่อยู่ของเพื่อนบ้านข้างเคียงไม่ได้ เว้นแต่ได้รับความยินยอม
"""
```

Typical RAG Workflow (1)

Step 1: Divide your documents into appropriately size/length

- find the appropriate chunk size based on your document
- If structure is important in your document, try using markdown dividers; they work well since they can be divided by the document's structure

Character splitter example



Markdown splitter example

For example, if we want to split this markdown:

```
md = '# Foo\n\n ## Bar\n\nHi this is Jim  \nHi this is Joe\n\n ## Baz\n\n Hi this is Molly'
```

We can specify the headers to split on:

```
[("#", "Header 1"),("##", "Header 2")]
```

And content is grouped or split by common headers:

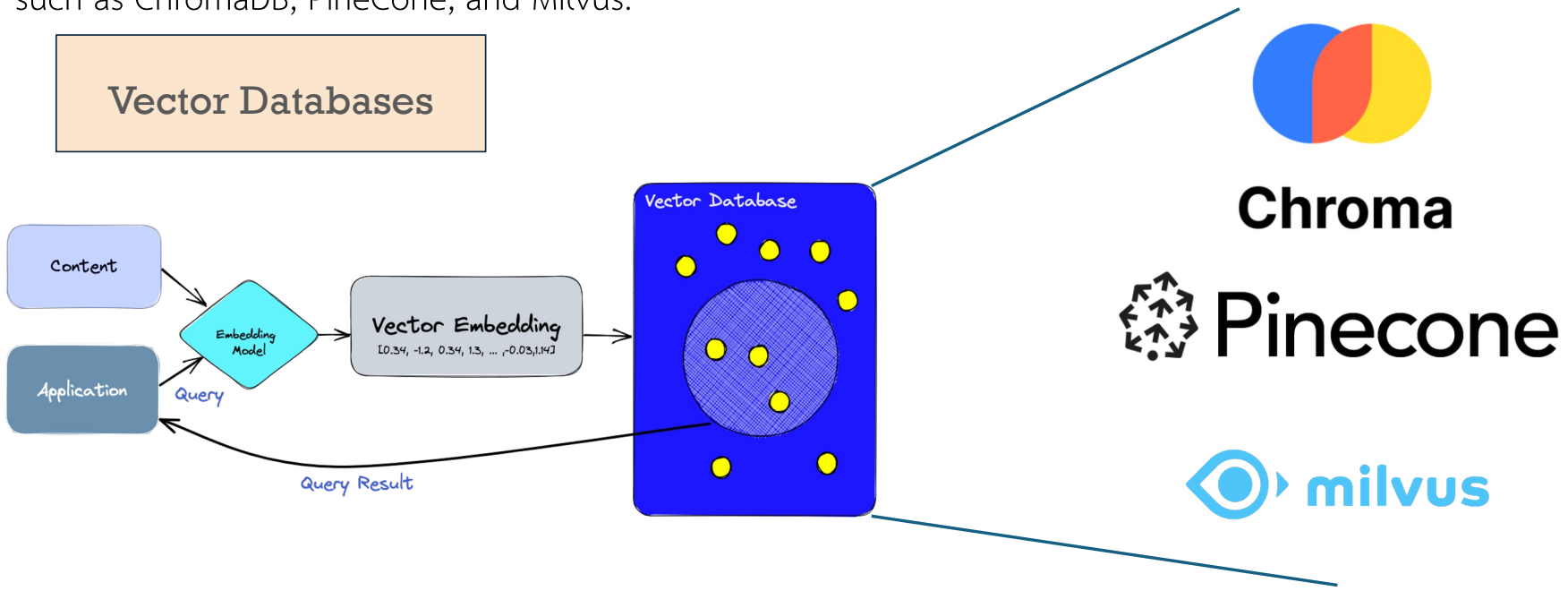
```
{'content': 'Hi this is Jim  \nHi this is Joe', 'metadata': {'Header 1': 'Foo', 'Header 2': 'Bar'}}
{'content': 'Hi this is Molly', 'metadata': {'Header 1': 'Foo', 'Header 2': 'Baz'}}
```


https://python.langchain.com/v0.1/docs/modules/data_connection/document_transformers/

Typical RAG Workflow (2) – Vector Database

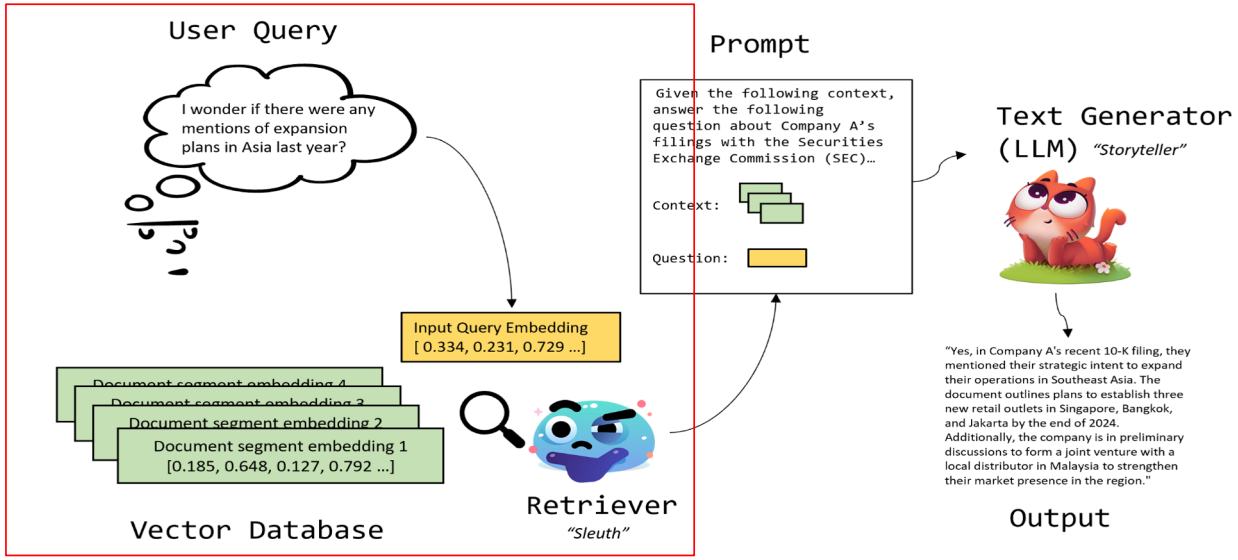
Step 2: Store the data based on usage requirements

The most popular RAG data sources are Vector Databases, which are currently offered by many services such as ChromaDB, PineCone, and Milvus.



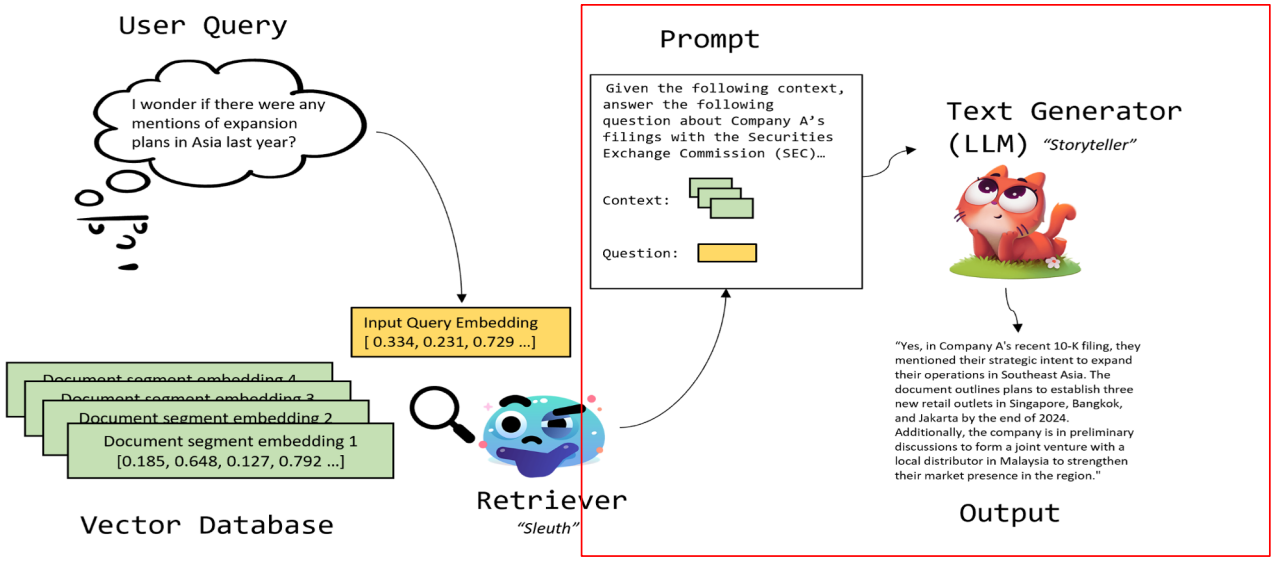
Typical RAG Workflow (3)

Step 3: Whether a user asks a question, the system converts the text into an embedding vector to find the document closest to the question, which is then used as context for the LLM.



Typical RAG Workflow (4)

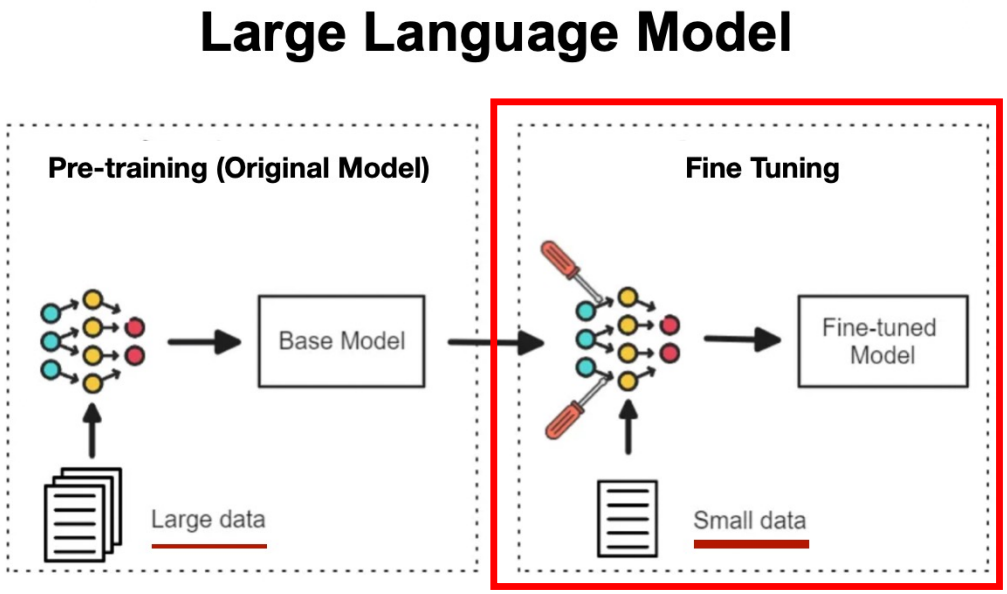
Step 4: The question text + context obtained from vector search are fed to the LLM to generate an answer. Therefore, the answer obtained will depend on the context provided, resulting in more reliable results because the answer is based on the context from the documents we input.



3. Fine-tuning

Fine-tuning

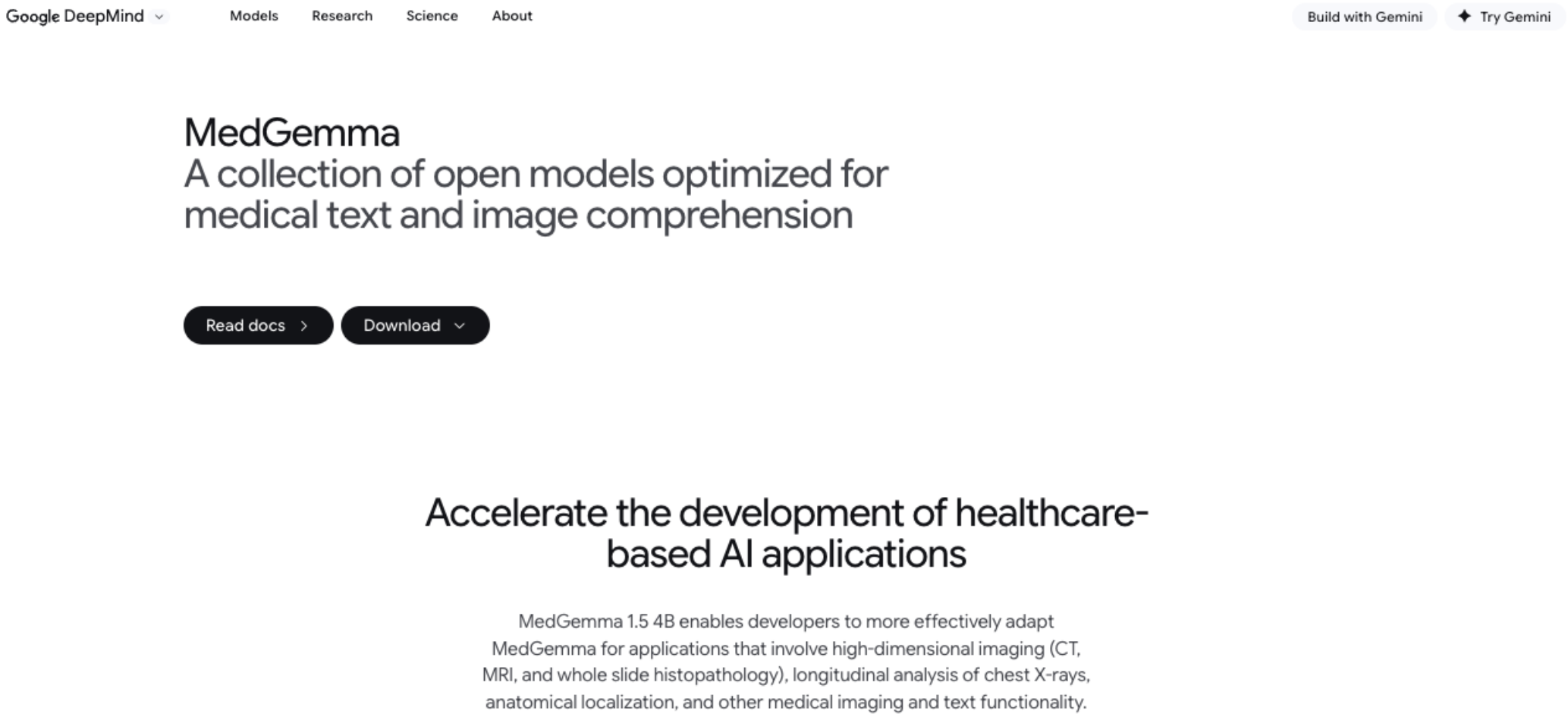
Fine-tuning is retraining a pre-trained LLM with your own domain-specific data. The goal is to make the model behave like an expert in a specific domain. It helps the model better understand specialized language, style, and tasks.



Fine-tuning vs. RAG

Fine-tuning improves how the model behaves, while RAG improves what the model knows.

Topic	Fine-tuning	RAG
Cost & Speed	Expensive to train, fast to run	Cheap to train, slower responses
Maintenance	Needs retraining when adding new data	Update data only
Knowledge Handling	Knowledge inside the model	Knowledge outside the model
Hallucination Risk	Lower in trained domain	Higher if retrieval techniques/tools is weak
Best For	Stable, repeated tasks	Changing, large knowledge

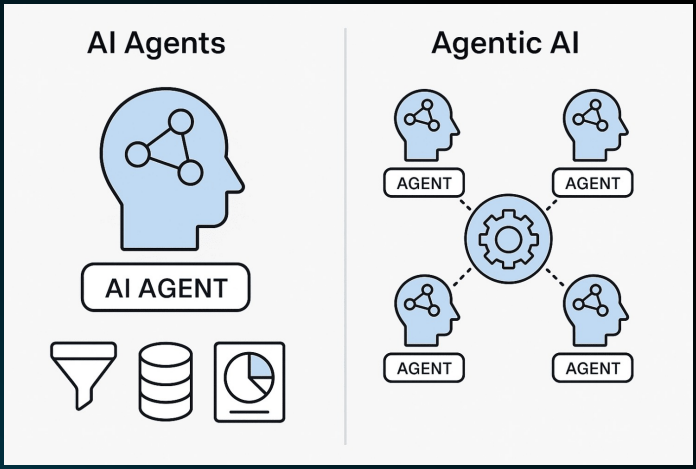


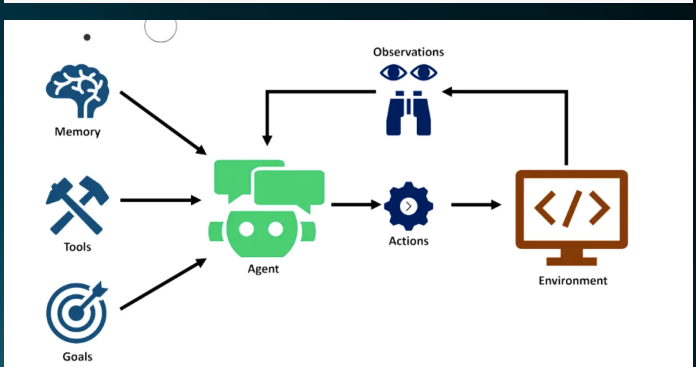
https://deepmind.google/models/gemma/medgemma/

4. Agentic AI

From Just Generative AI to Agentic AI

ReAct (Goal) = Reasoning (Thought) + Action + Observe





Published as a conference paper at ICLR 2023

REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS

Shunyu Yao[*,1], Jeffrey Zhao[2], Dian Yu[2], Nan Du[2], Izhak Shafran[2], Karthik Narasimhan[1], Yuan Cao[2]

[1]Department of Computer Science, Princeton University
[2]Google Research, Brain team
[1]{shunyuy,karthikn}@princeton.edu
[2]{jeffreyzhao,dianyu,dunan,izhak,yuancao}@google.com

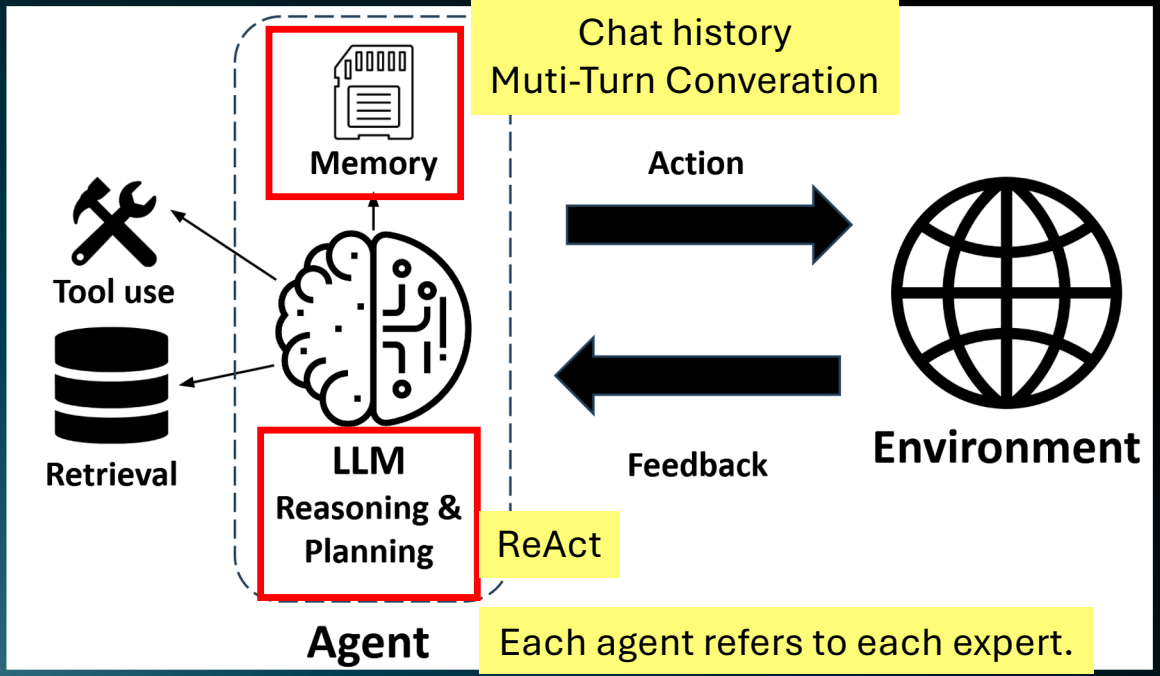
ABSTRACT

While large language models (LLMs) have demonstrated impressive performance across tasks in language understanding and interactive decision making, their abilities for reasoning (e.g. chain-of-thought prompting) and acting (e.g. action plan generation) have primarily been studied as separate topics. In this paper, we explore the use of LLMs to generate both reasoning traces and task-specific actions in an interleaved manner, allowing for greater synergy between the two: reasoning traces help the model induce, track, and update action plans as well as handle exceptions, while actions allow it to interface with and gather additional information from external sources such as knowledge bases or environments. We apply our approach, named ReAct, to a diverse set of language and decision making tasks and demonstrate its effectiveness over state-of-the-art baselines in addition to

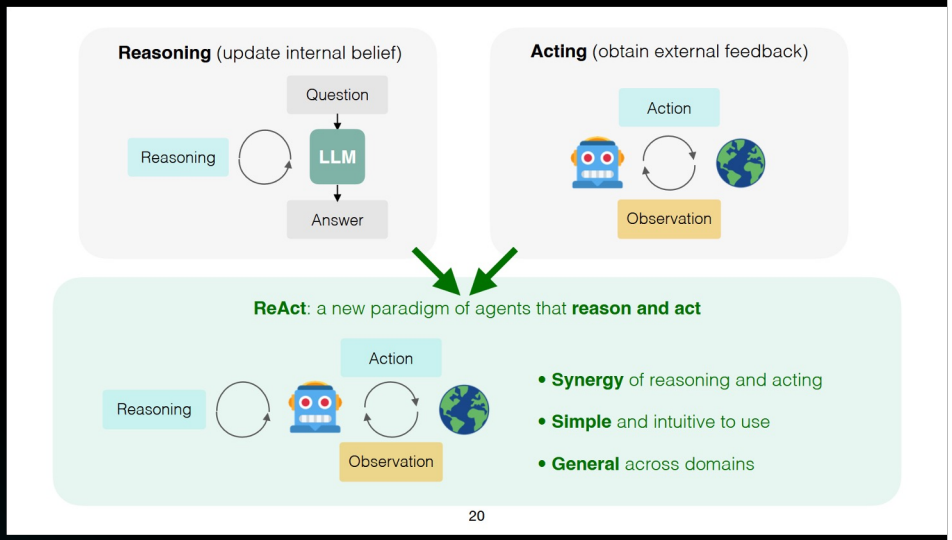
Agentic LLMs

ReAct = Reasoning (Thought) + Action + Observe

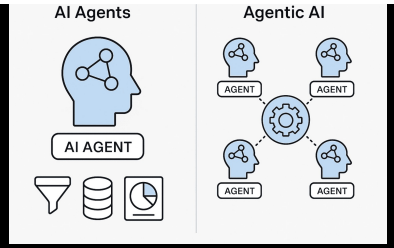
An agentic LLM is a large language model (LLM) that operates as an autonomous agent, capable of performing tasks, making decisions, and interacting with external tools or APIs to achieve the specified goals.



Reasoning: ReAct = Resoning + Action



Single Agents → Multiple Agents



Goal: "Find Q2 2025 sales revenue, compare it to Q2 2024, and report YoY growth."



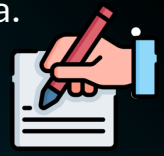
Different agents = different prompts

- **Step 1**
- Thought: I need Q2 2025 sales data first.
- Action: Call search_docs("Q2 Sales Report 2025")
- Observation: Found document with revenue = $1.45M

- **Step 2**
- Thought: To calculate growth, I also need Q2 2024 data.
- Action: Call search_docs("Q2 Sales Report 2024")
- Observation: Found document with revenue = $1.20M

- **Step 3**
- Thought: Now I can calculate YoY growth: (1.45M – 1.20M) ÷ 1.20M
- Action: Call calc_yoy(current=1.45M, last=1.20M)
- Observation: YoY growth = 20.8%

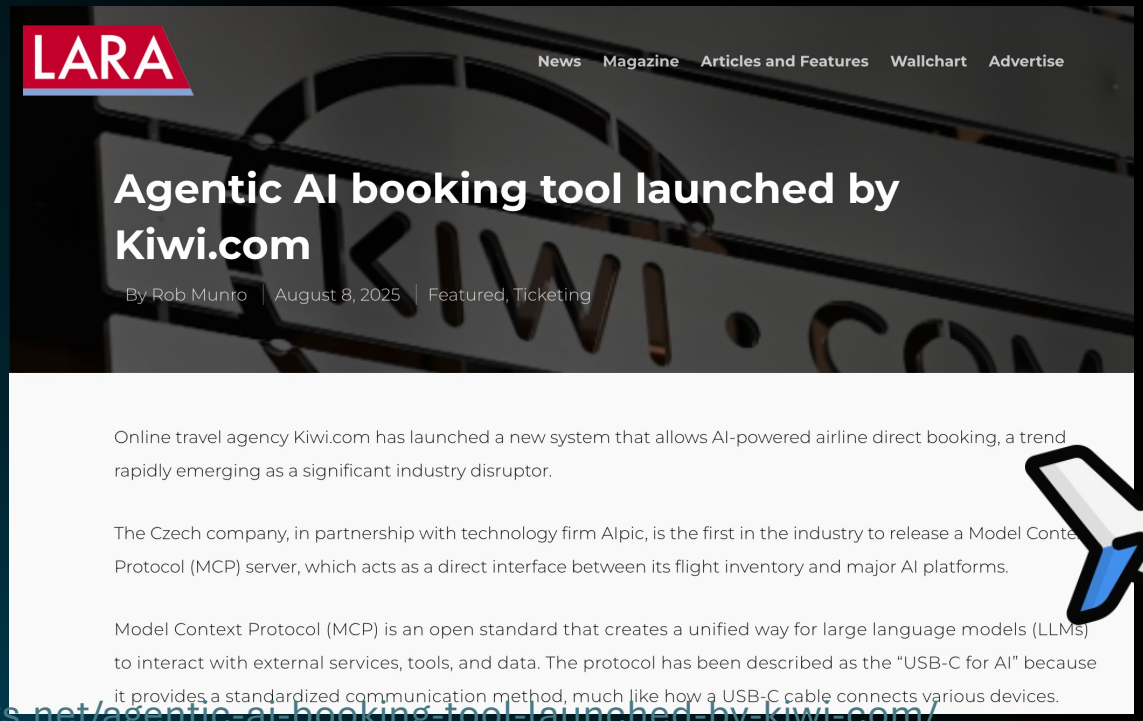
- **Step 4**
- Thought: I should summarize this result for the CFO.
- Action: Call summarize(revenue_now=1.45M, revenue_last=1.20M, yoy=20.8%)
- Observation: “Q2’25 revenue $1.45M vs Q2’24 $1.20M → YoY +20.8%”

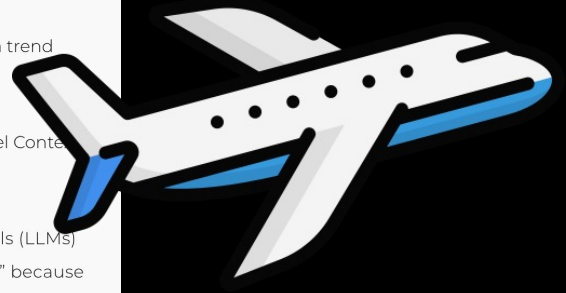


- **Step 5**
- Thought: Finally, I’ll draft an email with the summary.
- Action: Call draft_email(to="cfo@acme.com", subject="Q2 Update", body=summary)
- Observation: Email draft created, ready to send.

From Just Generative AI to Agentic AI
From Chatbot to Personal Assistant

- User: “Book me a round-trip flight from Bangkok to Tokyo next weekend, preferably morning flights, and send me the itinerary.”
- Agentic AI plans, searches, books, and delivers the itinerary automatically.



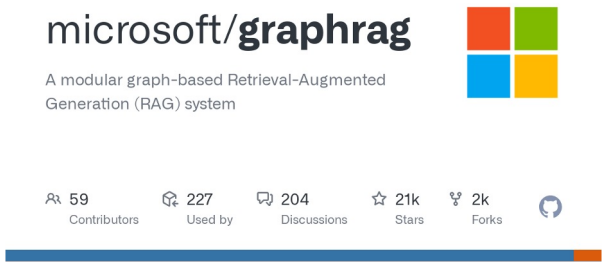


https://www.laranews.net/agentic-ai-booking-tool-launched-by-kiwi-com/

LLM Techniques

LLM Tools

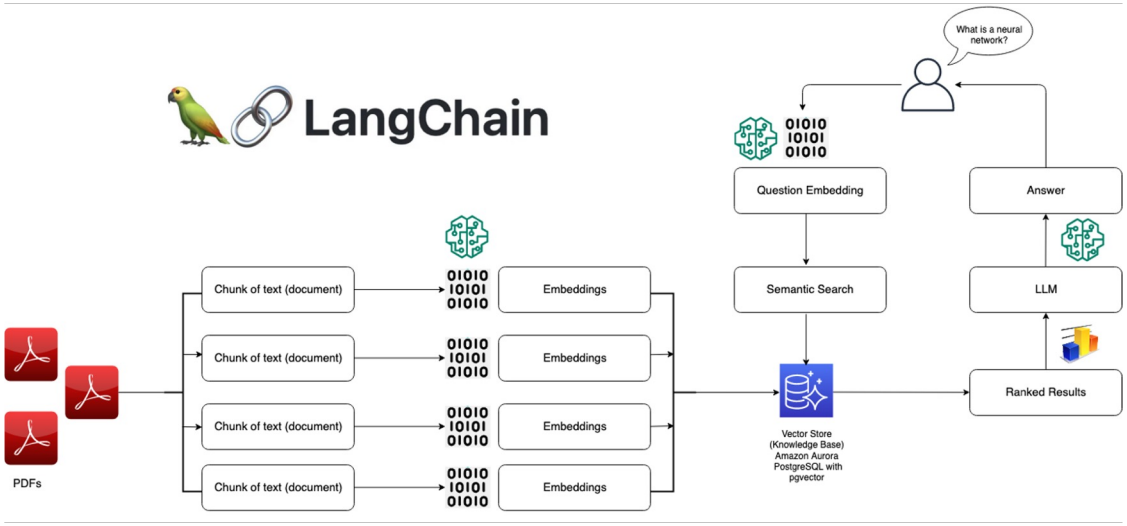
LLM Frameworks



Agentic Frameworks

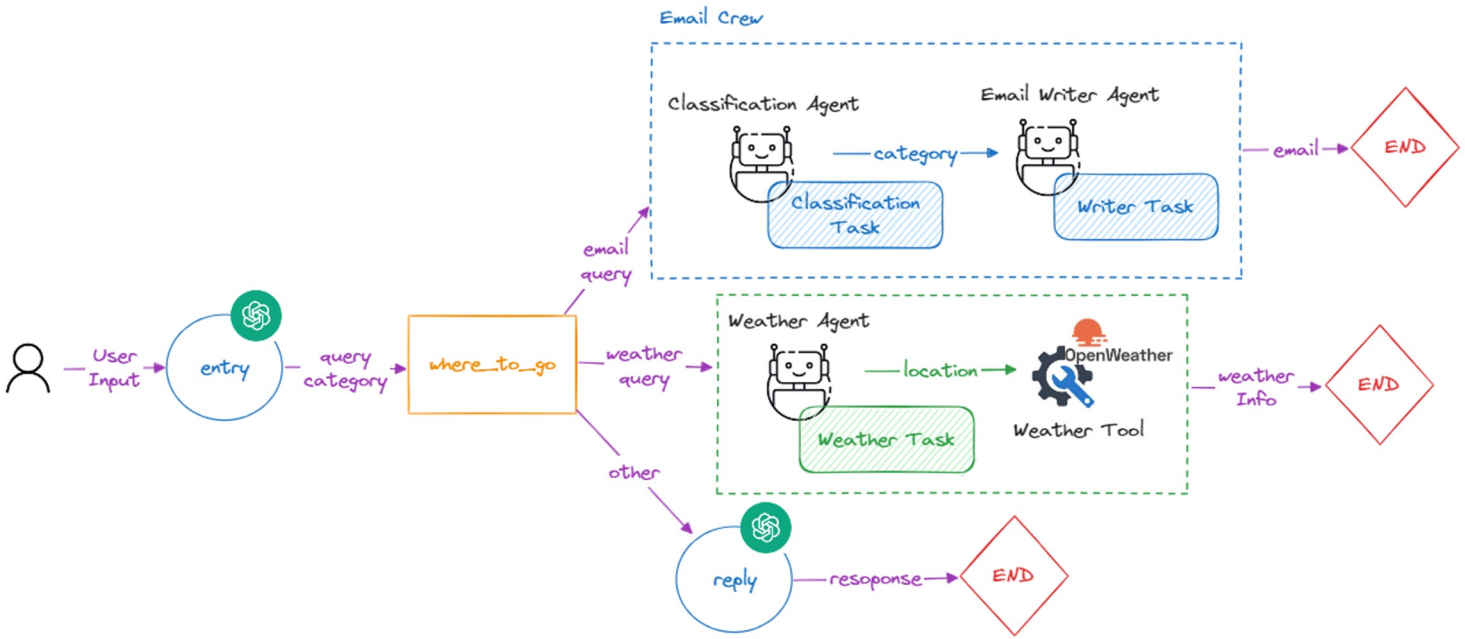
1) LangChain

LangChain is a framework that enables developers to build powerful LLM-based applications by managing external data connections, orchestrating multi-step interaction logic, and enhancing model capabilities such as context handling and tool execution, making it ideal for creating AI agents, chatbots, RAG systems, and complex automation workflows.



2) LangGraph

LangGraph is an orchestration framework for complex agentic systems and is more low-level and controllable than LangChain agents.



3) LangSmith (LLMOps)

Langsmith is designed with AI model debugging and orchestration in mind.

1) Observability

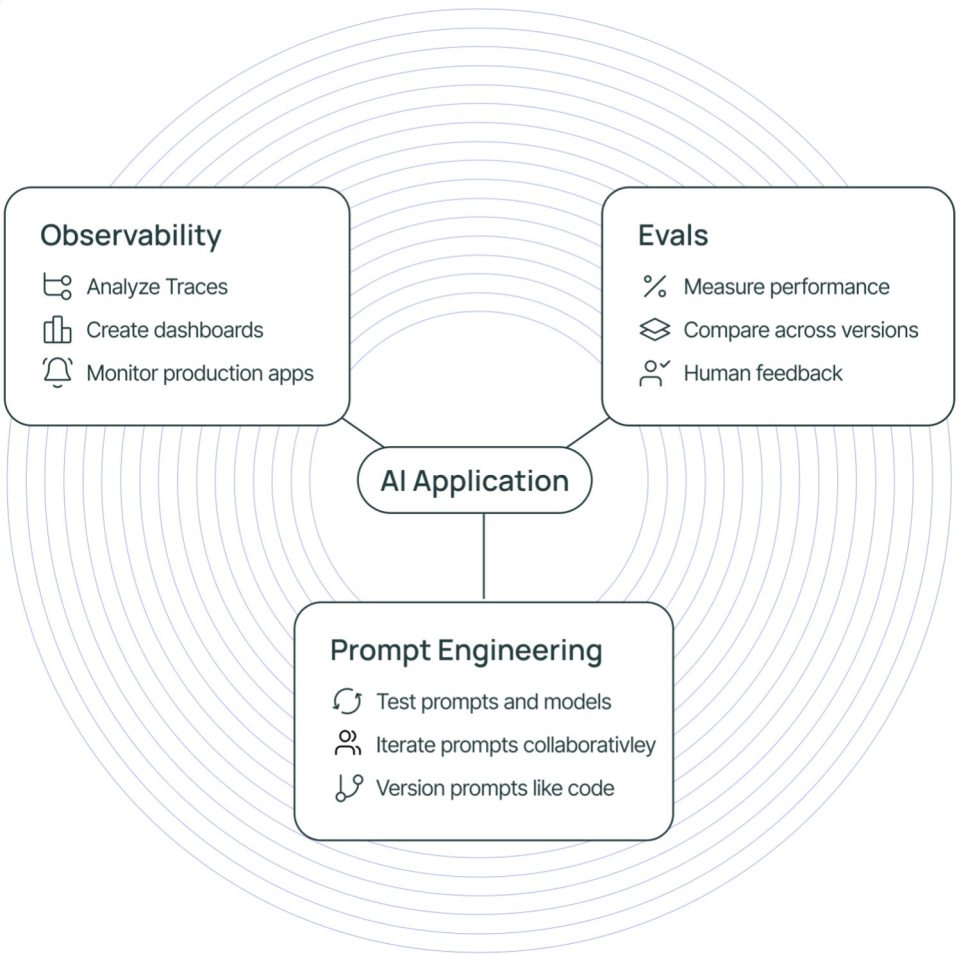
Analyze traces in LangSmith and configure metrics, dashboards, alerts based on these.

2) Evals

Evaluate your application over production traffic — score application performance and get human feedback on your data.

3) Prompt Engineering

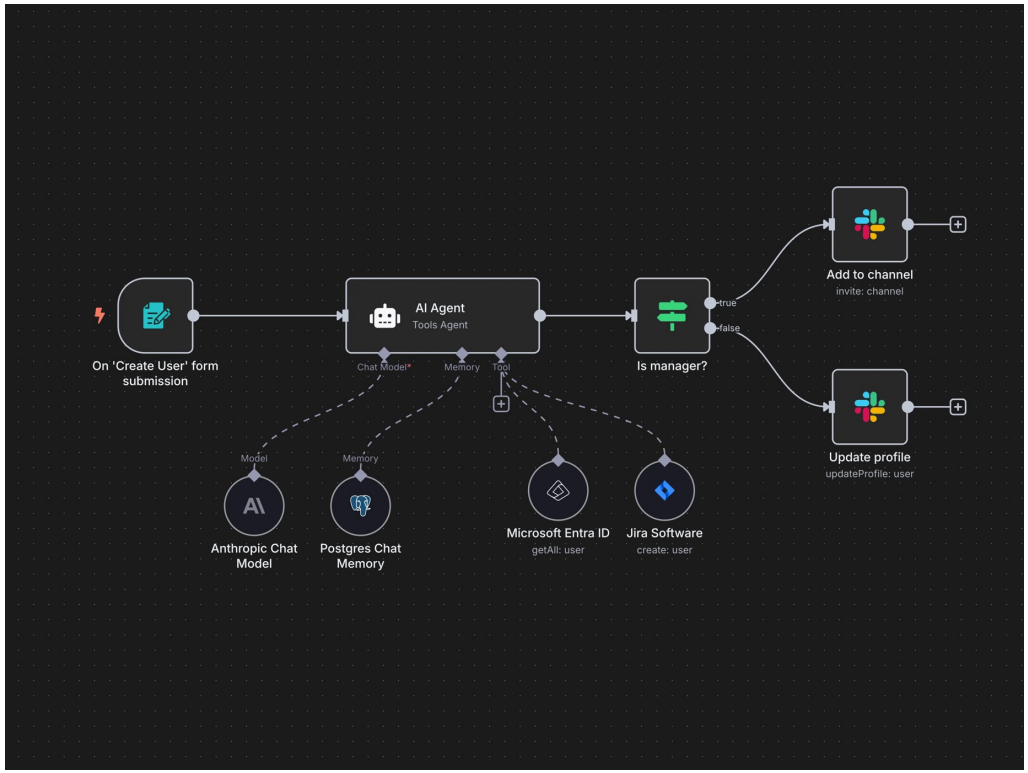
Iterate on prompts, with automatic version control and collaboration features.



https://www.langchain.com/langgraph

4) N8N

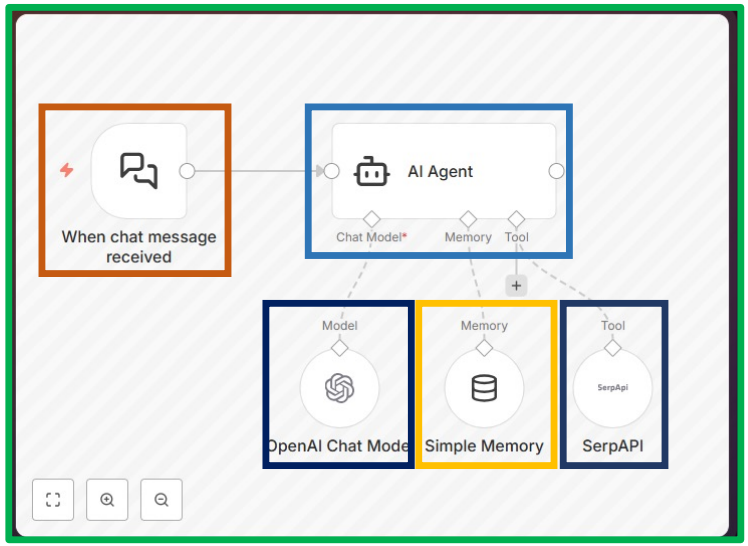
n8n (pronounced “n-eight-n”) is a workflow automation platform that helps you connect apps, move data, and automate tasks without needing to write a lot of code. You can think of it as a visual tool for automating repetitive work by designing flows that link different services together.



https://n8n.io/

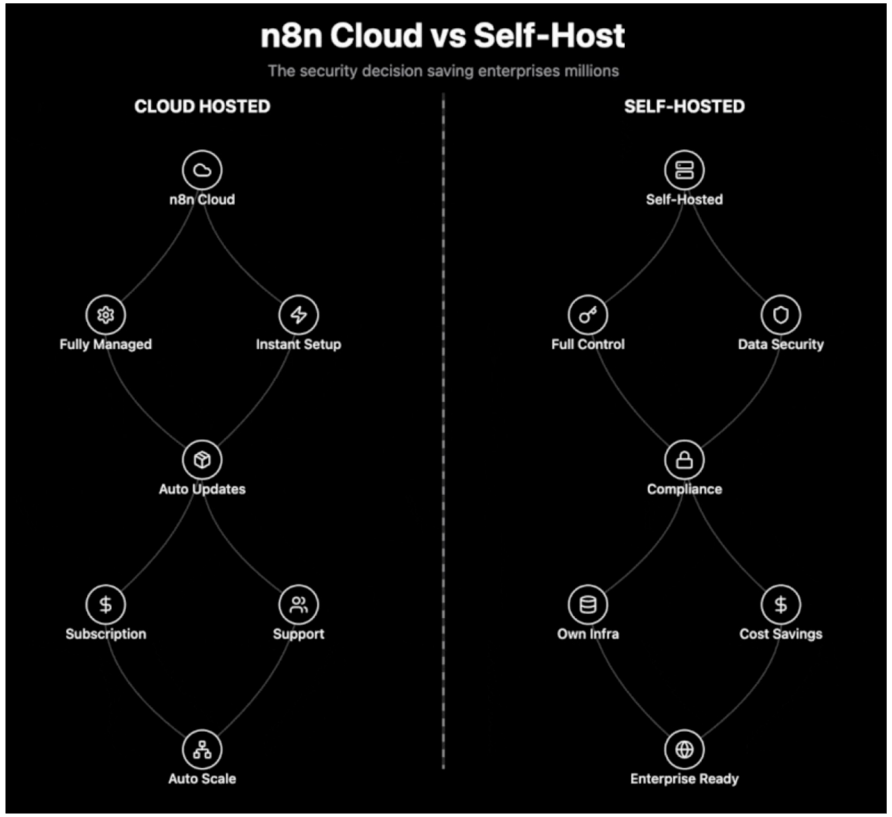
N8N Components

1. Workflow: The complete automation that connects all nodes to handle a chat message from start to finish.
2. Node: A single functional step in the workflow that performs a specific action. In n8n there are a lot of node types. for example,
    1. Trigger Node: The entry point of the workflow that starts execution when get notified (such as when receive messages).
    2. AI Agent Node: The main orchestration node that processes user input and orchestrates models, memory, and tools.
    3. Memory Node: A sub-node attached to the AI Agent that stores and retrieves conversational context.
    4. Chat Model Node: LLM/Chat model node that will receive input and get generative response.
3. Tools: External capabilities (e.g., search or APIs) that the AI Agent can call to enrich its response.



https://n8n.io/workflows/1954-ai-agent-chat/

Where to use?

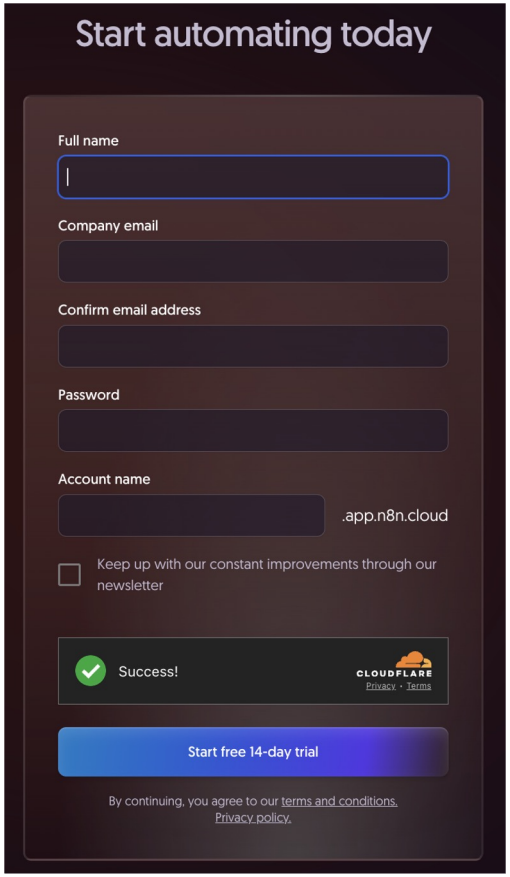


1) Localhost (your computer) - setup require Docker/Node.js

2) Self-hosted (your own server) - you are responsible for **deployment**, security, updates, and uptime

✅**3) n8n Cloud (managed service) — used in this lab**

- n8n hosts and maintains the platform for you (no server setup)
- Best for: fastest start, reliable uptime, easy sharing/collaboration
- **In this lab:** use the **14-day free trial** to build workflows without installation



Registration & Free Trial Activation (n8n Cloud)

Go here.

Let’s Drag, Drop, and Automate

What we will build in n8n (using the built-in Chat UI)

Lab 8.0 — Simple Q&A Chatbot

Lab 8.1 — Medical Research Assistant with PubMed

- Input: a natural-language question in chat
- Workflow: validate the request → create a structured PubMed search plan → call PubMed APIs → format a concise answer with citations (PMID/title/journal/year) → respond in chat.

Lab 8.2 — Document-based RAG Chatbot

- Input: either (a) upload a document, or (b) ask a question
- Workflow:
    - Upload path: load document → chunk text → create embeddings → store in a vector database
    - Question path: retrieve relevant chunks → generate an answer grounded in retrieved text → respond in chat.

+ Thank you
& any questions